



가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 06 - Road Networks and Traffic

Chris Ohk

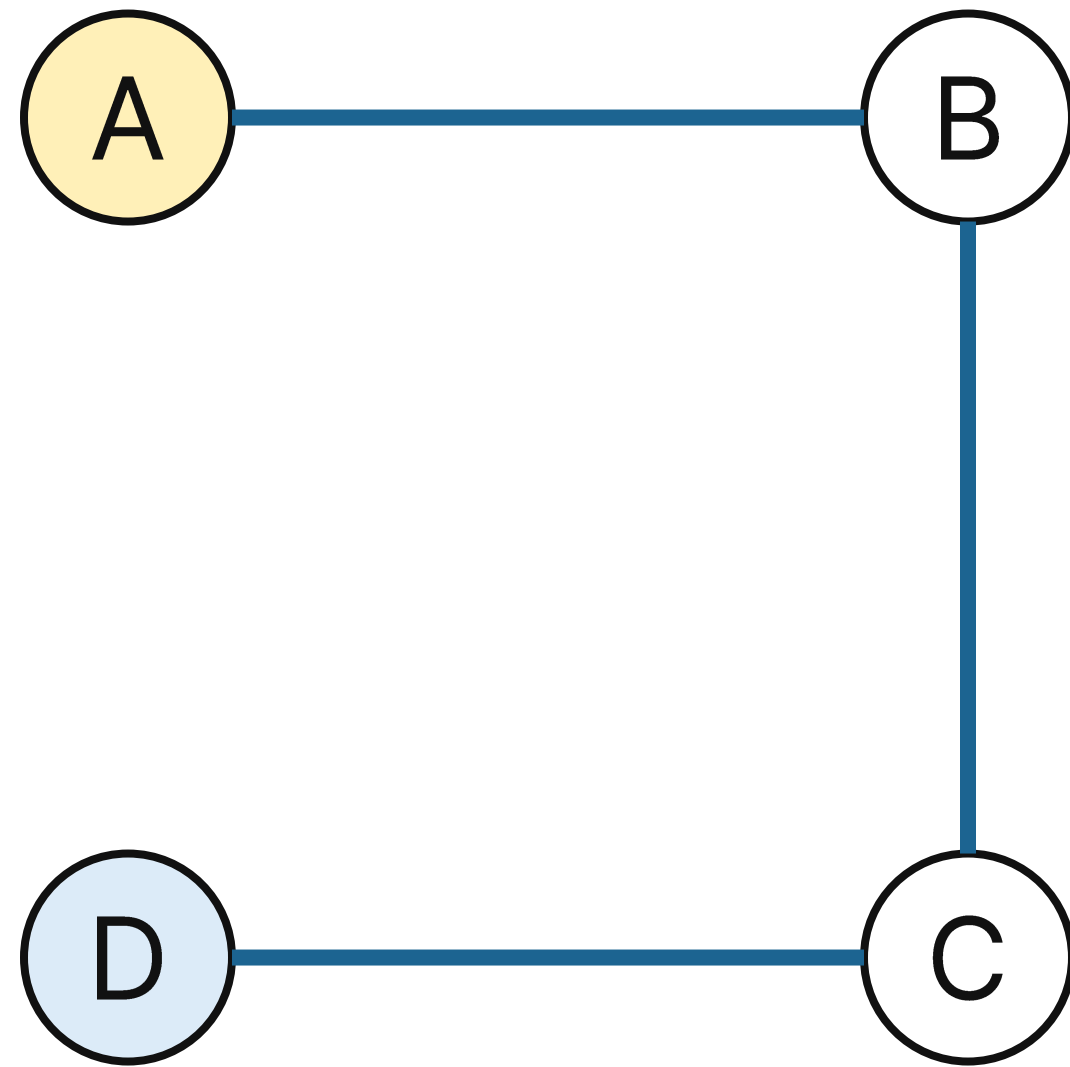
utilForever@gmail.com

- 건물 옆에 도로가 있는지 확인했다.
 - 상하좌우 이웃에 도로 셀이 하나라도 있으면 조건을 만족했다.
- 이번에는 목적지까지 이어지는 도로를 확인한다.
 - 도로가 옆에 있어도 직장이나 병원으로 갈 수 없을 수 있다.

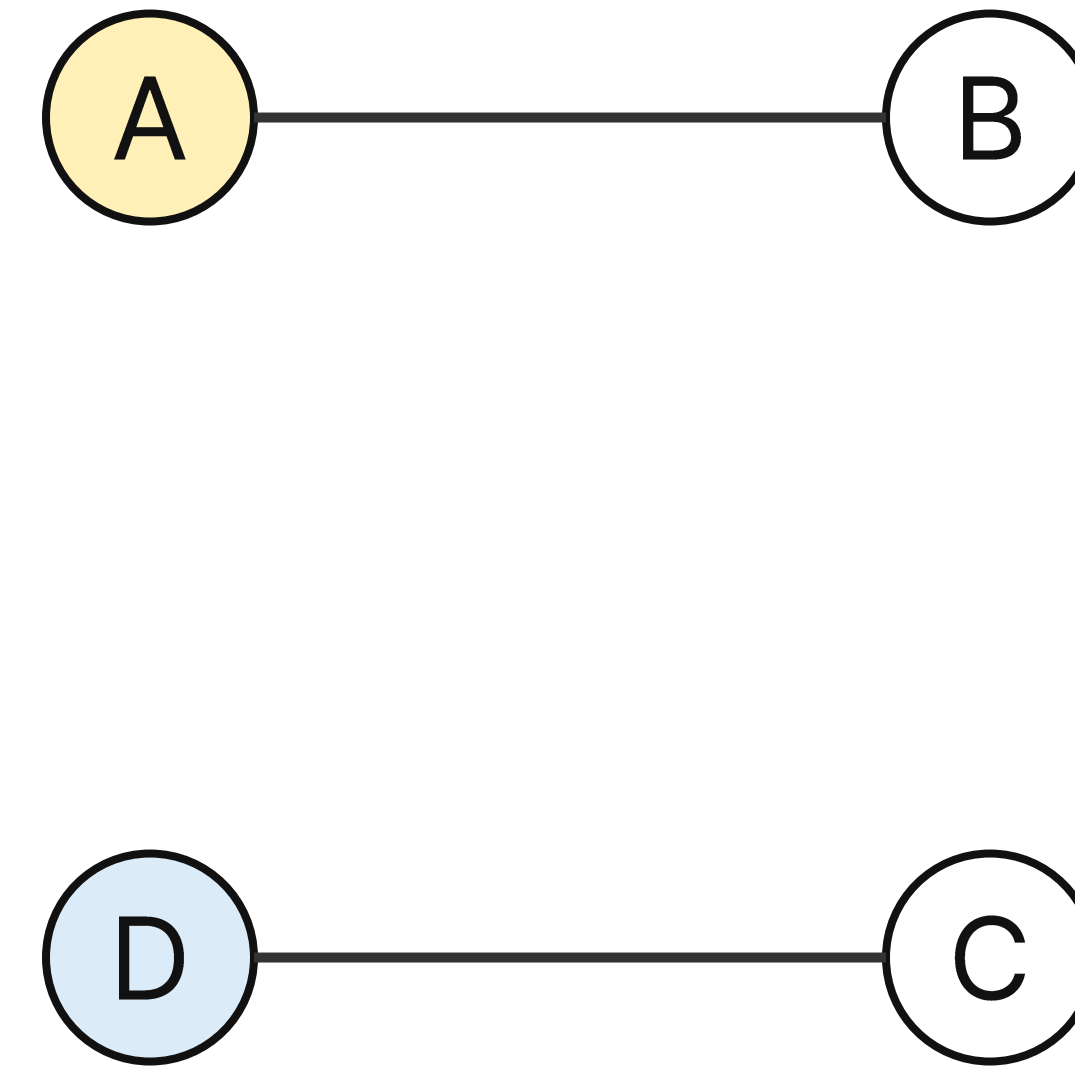
- 주택에서 직장 and 병원까지 이동할 수 있는가?
 - 연결된 도로를 찾고 이동할 경로를 선택한다.
- 가장 가까운 길은 가장 빠른 길인가?
 - 거리와 이동시간을 서로 다른 비용으로 계산한다.
- 도로가 막히면 도시의 접근성은 어떻게 달라지는가?

위치 정보만으로는 부족하다

- 출발지와 목적지의 위치만으로는 이동 가능 여부를 알 수 없다.
 - 출발지 : A, 목적지 : D
 - 중간에 도로가 끊기면 목적지에 갈 수 없다.



도로가 이어진 경우



중간에 도로가 끊긴 경우

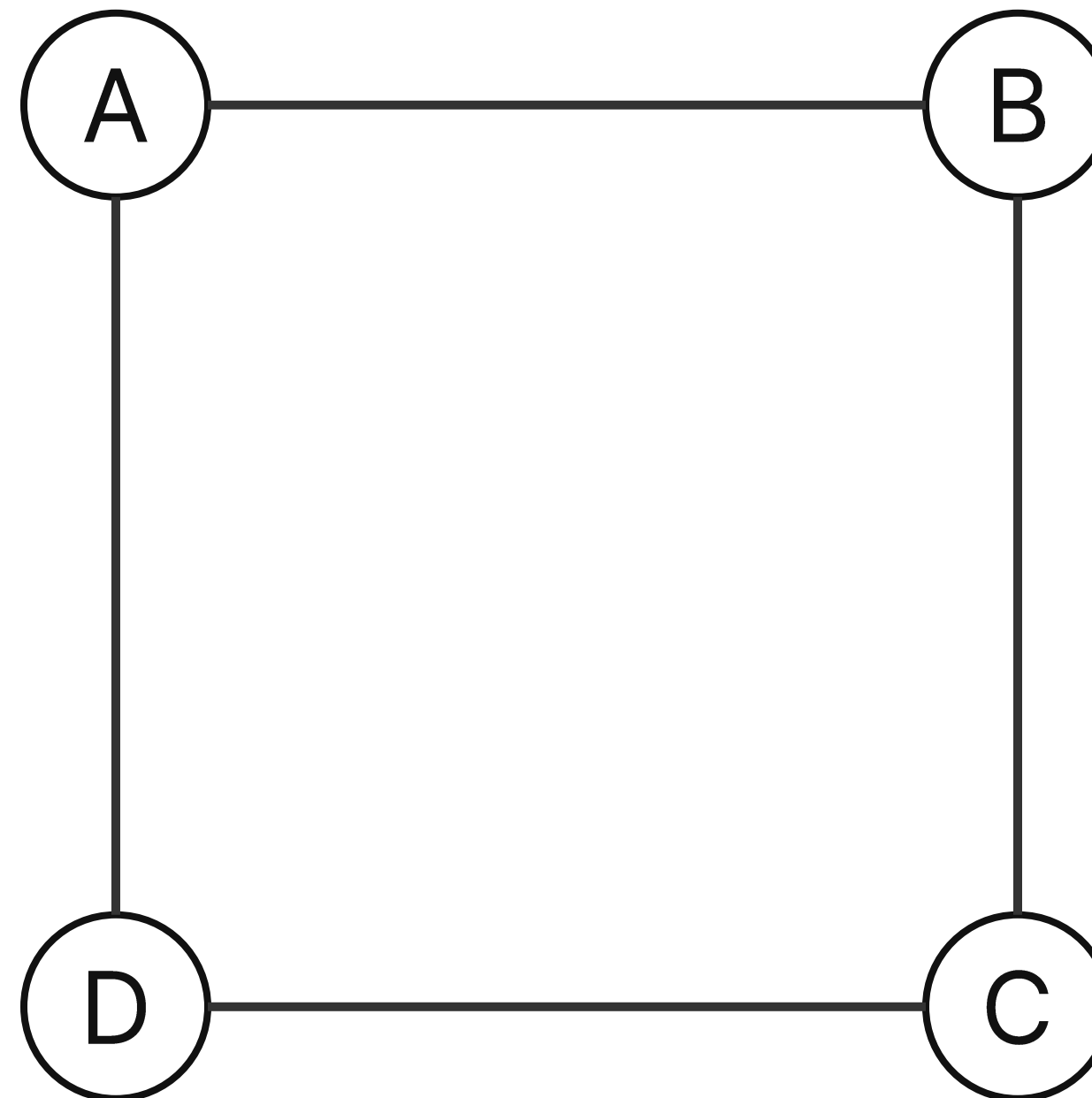
그래프가 필요한 이유

- 이동을 계산하려면 장소 사이의 연결 관계를 표현해야 한다.
 - 각 장소에서 바로 이동할 수 있는 곳을 기록한다.
- 연결을 차례로 따라가며 경로를 찾는다.
 - 도달 가능 여부 / 이동 경로 / 거리와 시간을 계산할 수 있다.
- 이 연결 관계를 다루는 기본 모형이 그래프graph다.

- 그래프는 대상과 대상 사이의 관계를 표현하는 자료구조다.
 - 대상은 정점vertex 또는 노드node로 나타낸다.
 - 관계는 간선edge으로 나타낸다.
- 도로망에서는 위치와 이동 가능 관계를 표현한다.
 - 교차로를 노드로 정할 수도 있고, 도로 셀을 노드로 정할 수도 있다.

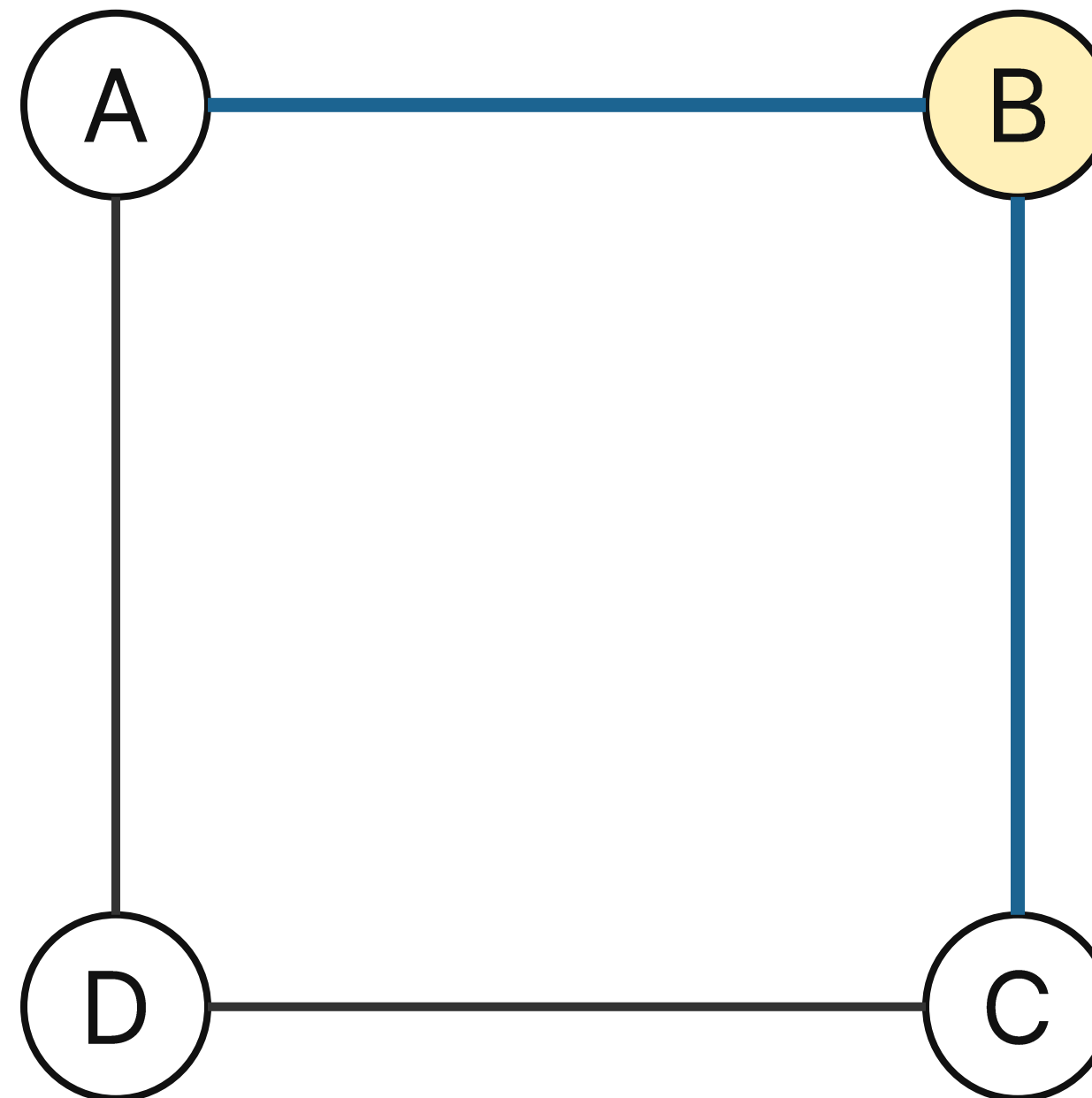
노드 집합과 간선 집합

- 그래프를 $G = (V, E)$ 로 나타낸다.
 - 노드 집합 $V = \{A, B, C, D\}$
 - 간선 집합 $E = \{\{A, B\}, \{B, C\}, \{C, D\}, \{D, A\}\}$

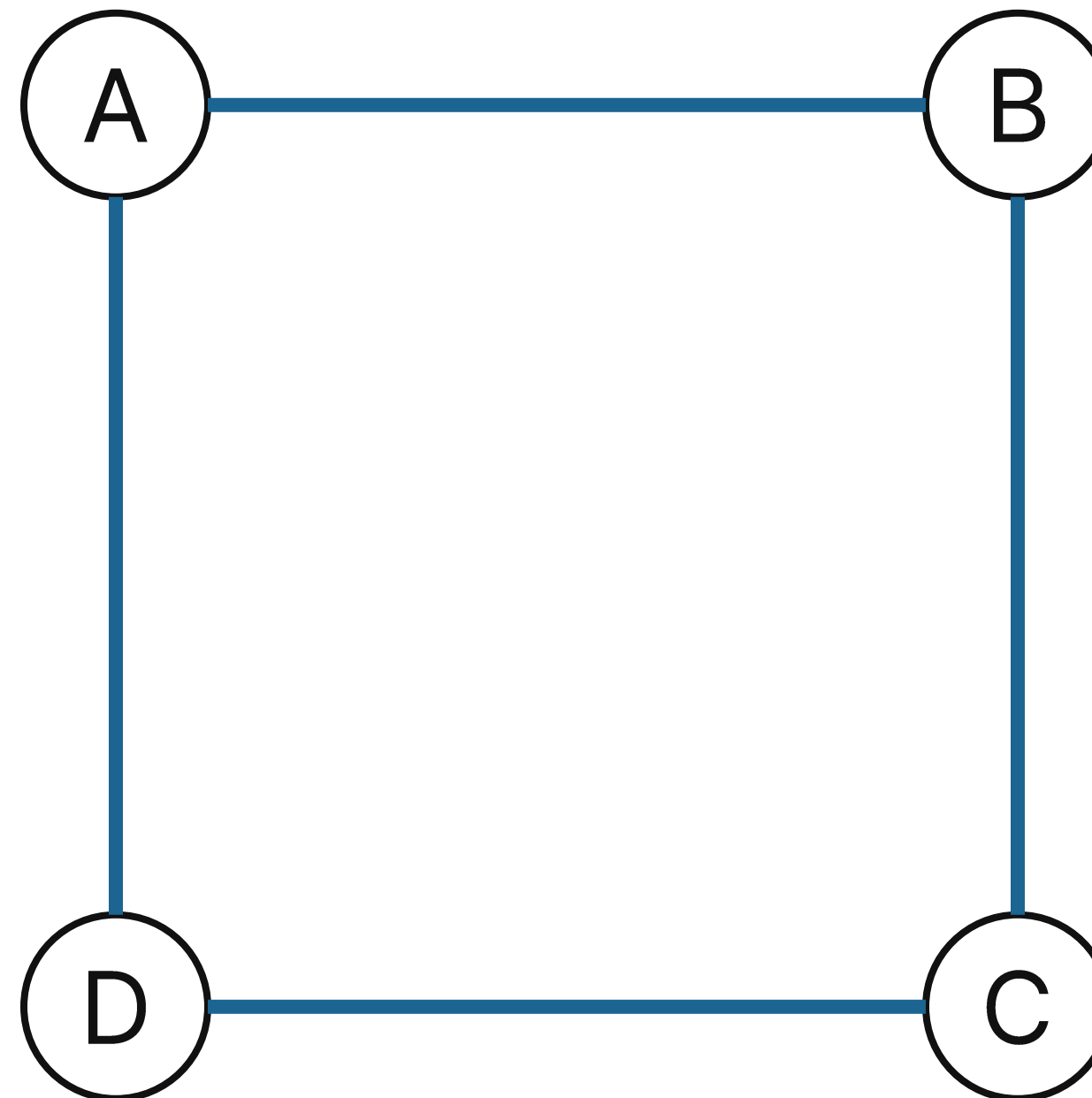


인접한 노드와 차수

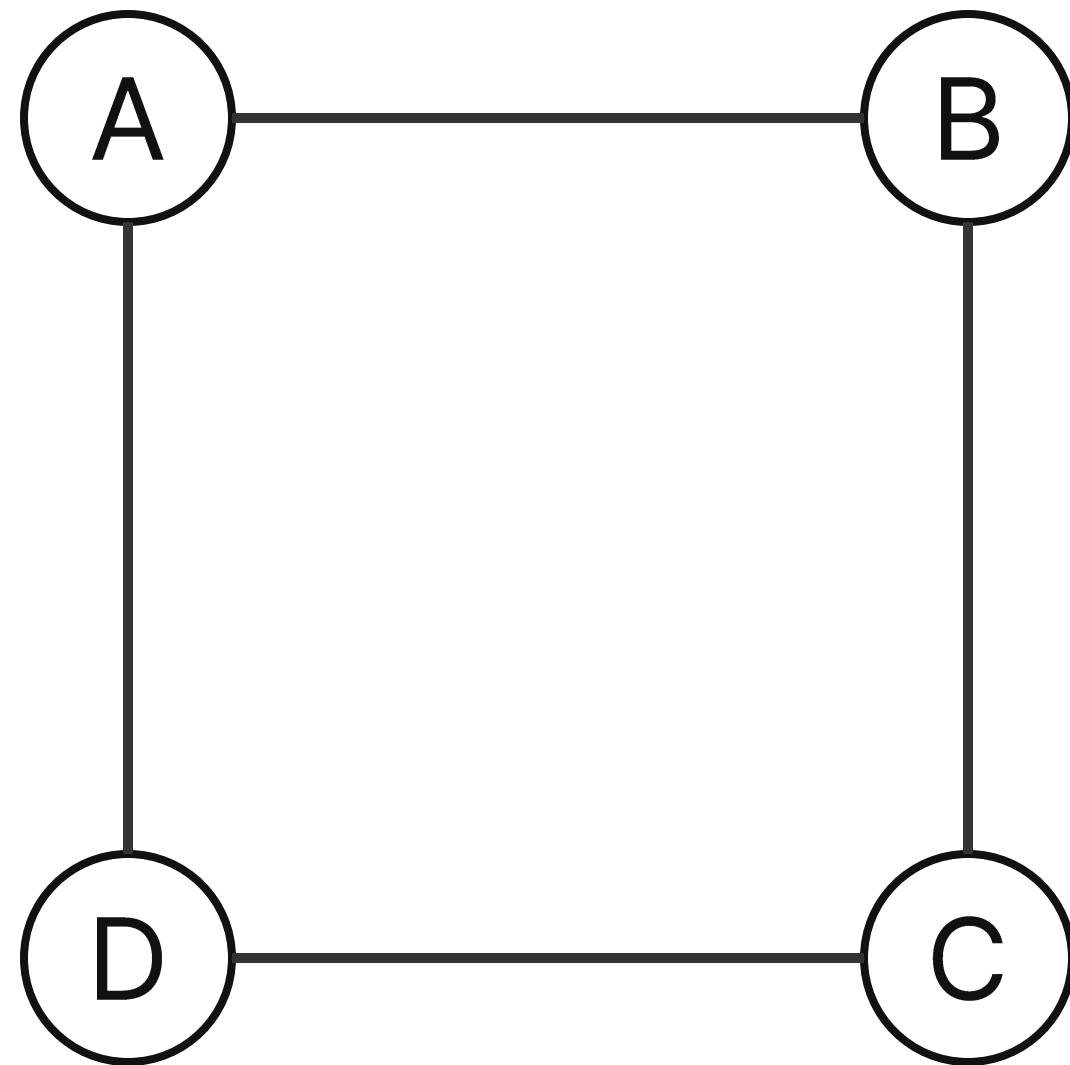
- 간선 하나로 직접 연결된 두 노드를 인접하다 adjacent고 한다.
- B의 이웃은 A와 C
- B에 연결된 간선 수인 차수 degree는 2다.



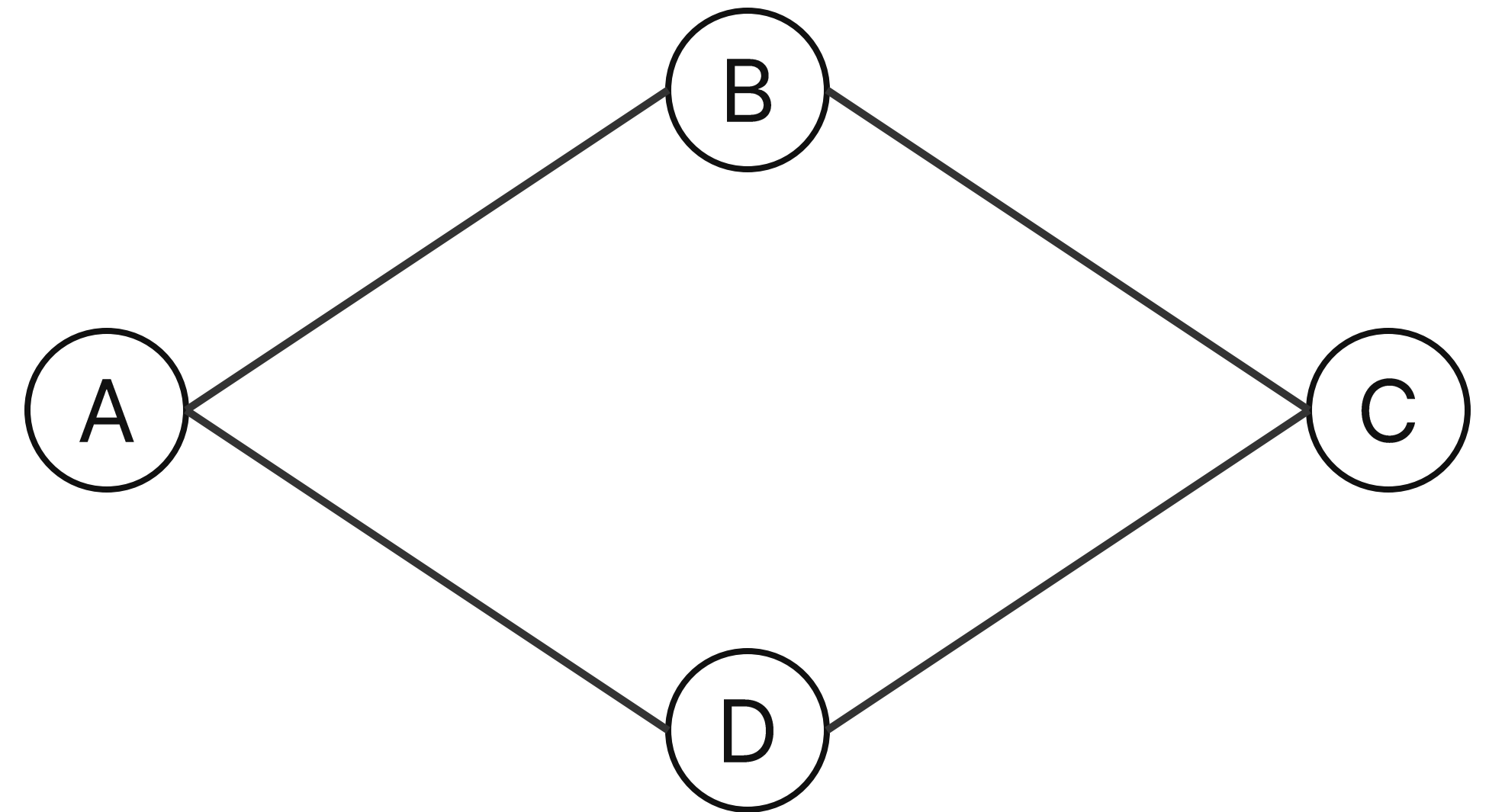
- 경로는 간선을 따라 이어지는 노드의 순서다.
- $A - B - C$ 는 간선 2개를 지나는 경로path다.
- $A - B - C - D - A$ 처럼 한 정점에서 시작해 간선들을 따라 다시 원래의 정점으로 돌아오는 길이 있는 경우 순환cycle이 있다고 한다.



- 같은 연결 관계는 다른 모양으로 그려도 같은 그래프다.



지도에 가까운 배치

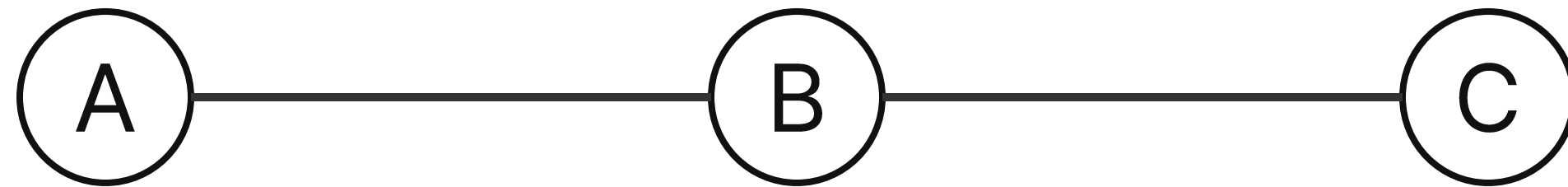


연결 관계를 강조한 배치

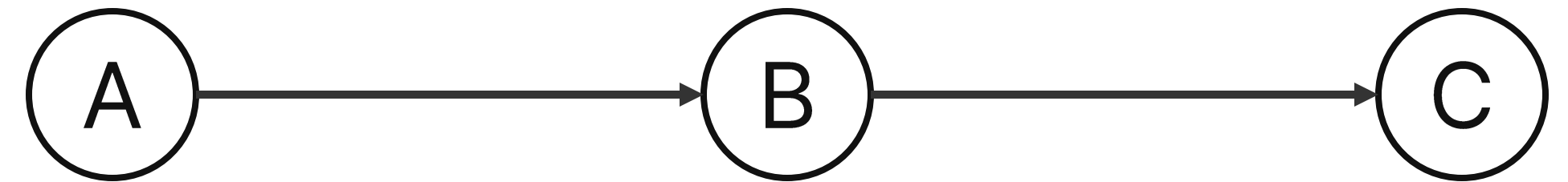
무방향 그래프와 방향 그래프

가상공간과 메타버스
Road Networks and Traffic

- 양방향 연결은 선으로, 허용된 방향이 있는 연결은 화살표로 표현한다.



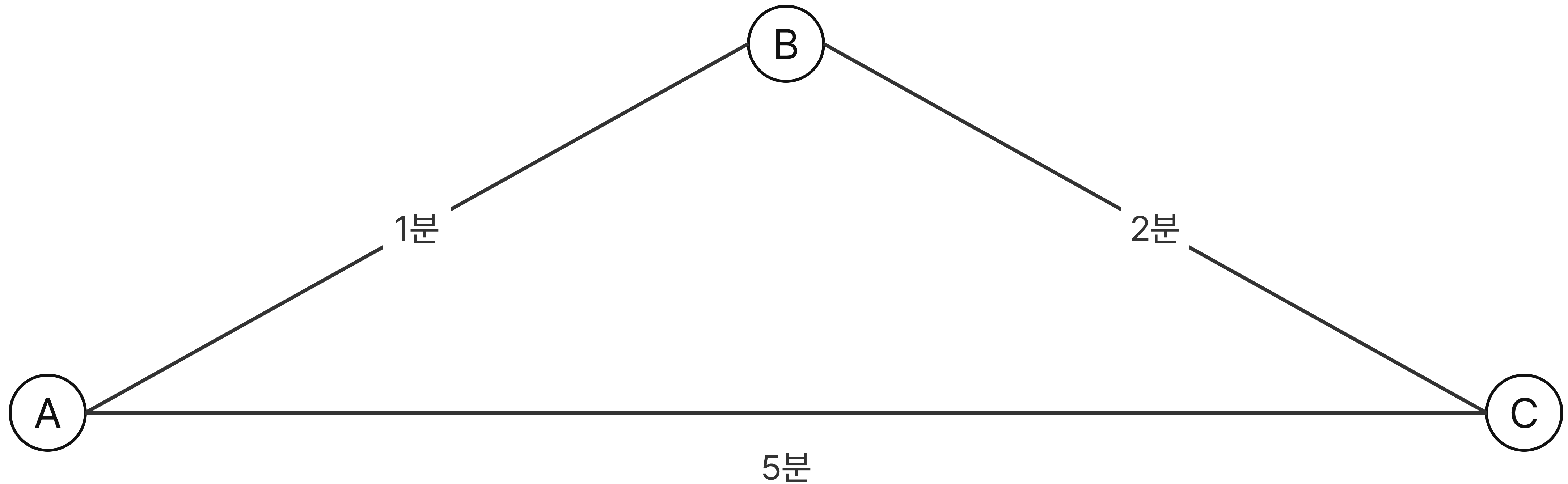
무방향 / 양방향 이동



방향 있음 / 화살표 방향으로 이동

간선의 가중치

- 가중치는 간선을 지날 때 드는 비용이다.
- 아래 그래프에서는 이동시간을 분 단위로 표시했다.



- 도로 셀 하나를 노드 하나로 표현한다.
 - 상하좌우에 있는 도로 셀끼리 이동할 수 있다.
- 예제의 도로는 모두 양방향이다.
 - 대각선 이동 / 높이 차이 / 차선 변경은 다루지 않는다.
- 셀 중심 간격은 100m이며, 기본 이동시간은 간선당 1분이다.

- 회색 셀은 도로이며 A부터 I까지 이름을 붙였다.
- 주택은 A / 직장은 G / 병원은 H에서 도로와 연결된다.

y / x	0	1	2	3	4	5	6
0	주택						
1	A	B	C			H	병원
2	D	E	F	G		I	
3				직장			

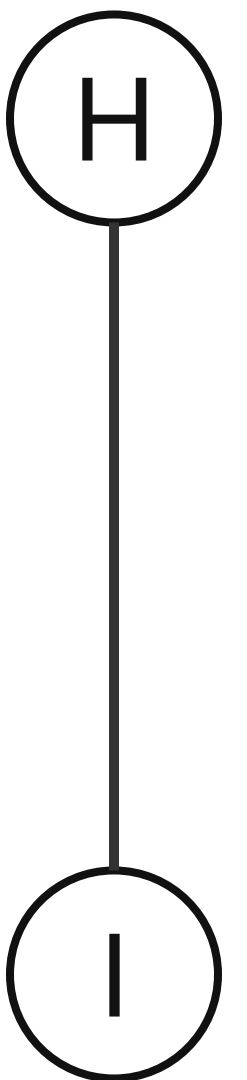
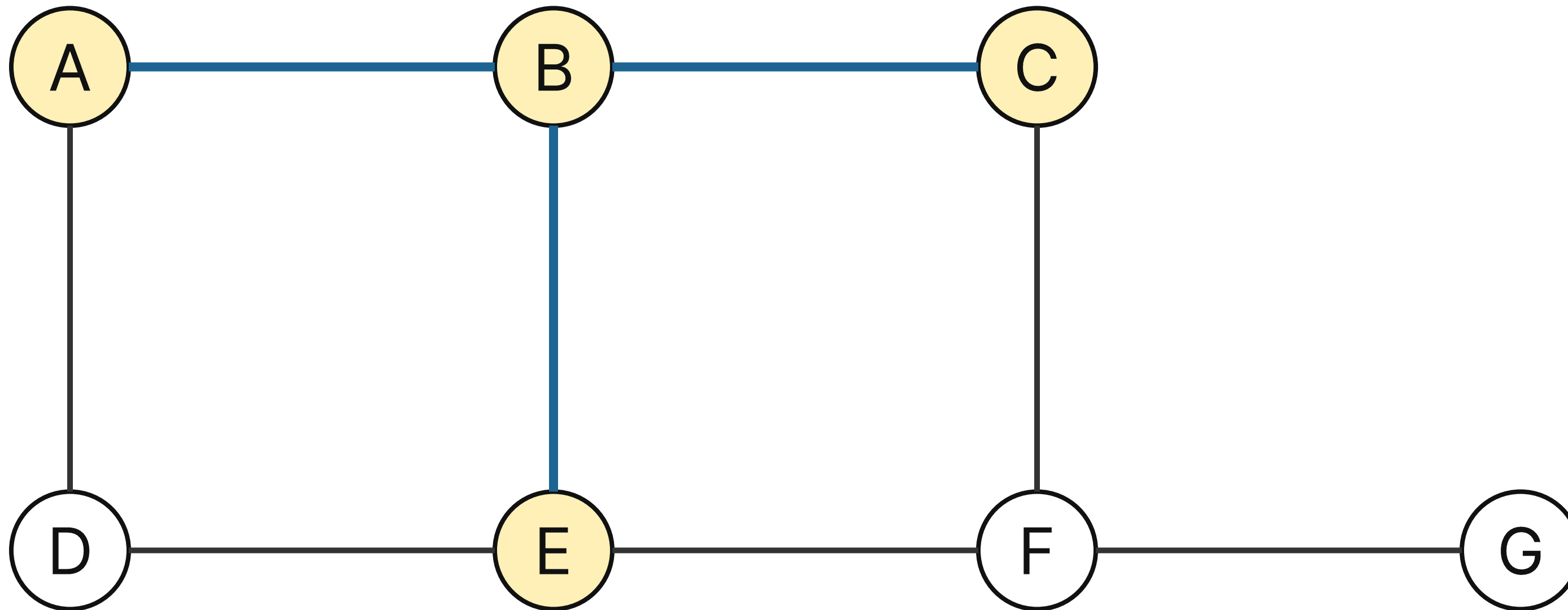
도로 셀의 노드 변환

- 도로인 셀마다 고유한 노드 ID를 만든다.
- 좌표 (x, y) 를 ID로 사용해도 된다.

ID	좌표	ID	좌표	ID	좌표
A	(0, 1)	D	(0, 2)	G	(3, 2)
B	(1, 1)	E	(1, 2)	H	(5, 1)
C	(2, 1)	F	(2, 2)	I	(5, 2)

이웃 셀의 간선 변환

- 각 도로 셀에서 상하좌우의 도로 셀을 연결한다.
- B의 이웃은 C / E / A이며, 대각선 F는 포함하지 않는다.



- 인접 리스트는 각 노드의 이웃 목록을 저장하는 방식이다.
- 딕셔너리의 키는 노드 / 값은 바로 이동할 수 있는 노드 목록이다.

노드 키	이웃 목록 값	의미
"A"	["B", "D"]	A에서 B 또는 D로 한 번 이동
"B"	["C", "E", "A"]	B에서 C / E / A로 한 번 이동
"G"	["F"]	G에서 F로 한 번 이동

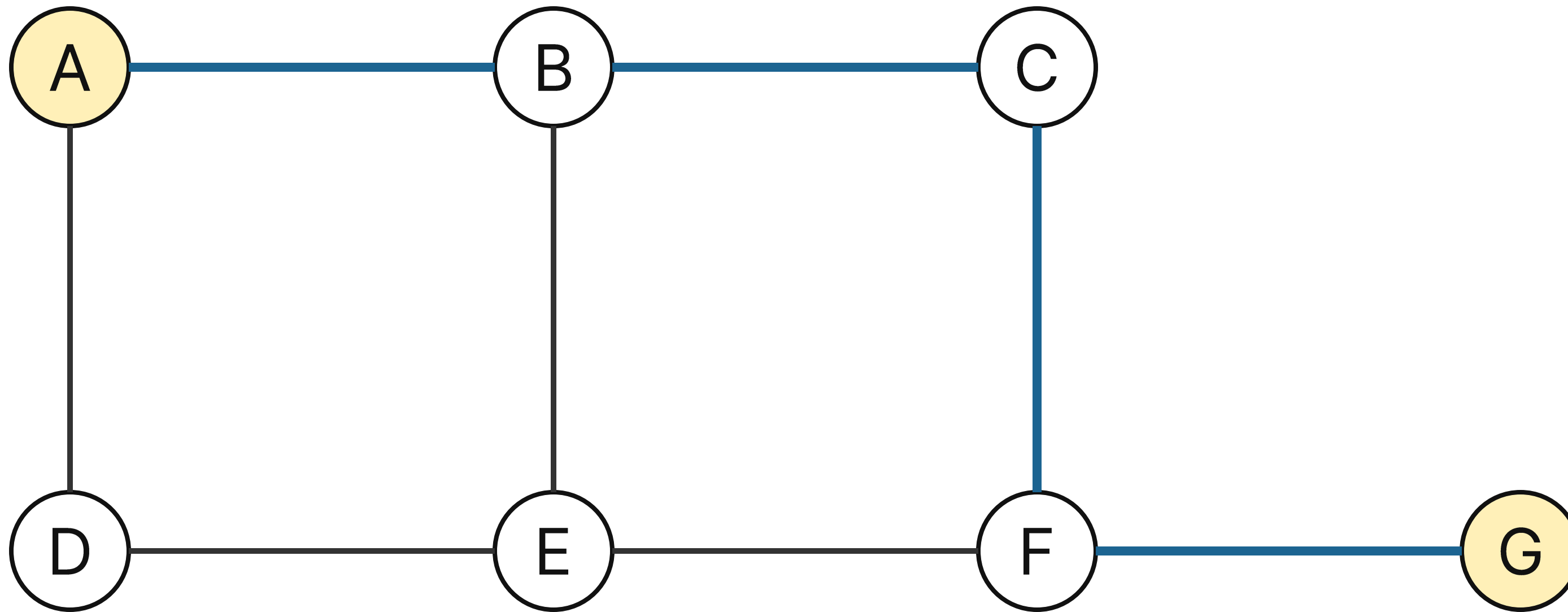
- 각 노드에서 바로 이동할 수 있는 노드를 저장한다.
- Python에서는 딕셔너리와 리스트로 표현할 수 있다.

노드	이웃 목록	노드	이웃 목록
A	[B, D]	F	[G, E, C]
B	[C, E, A]	G	[F]
C	[F, B]	H	[I]
D	[E, A]	I	[H]
E	[F, D, B]	—	—

- 노드 9개 / 무방향 간선 9개가 만들어진다.
 - 인접 리스트에는 각 간선이 양쪽에 한 번씩 기록된다.
- 도로를 편집하면 연결 관계도 갱신한다.
 - 도로를 지웠는데 이전 간선이 남아 있으면 잘못된 경로가 나온다.
- 작은 도시에서는 그래프 전체를 다시 만들어도 된다.

- 건물의 출입구와 연결되는 도로 노드를 정한다.
 - 주택은 A / 직장은 G / 병원은 H를 사용한다.
- 도로에 접한다고 해서 목적지까지 연결된다고 보장할 수는 없다.
 - 출발지의 건물 출입구에서 출발해 도로를 통해 목적지의 건물 출입구에 갈 수 있어야 한다.
 - 건물 출입구가 여러 개라면 어느 출입구를 쓸지도 정해야 한다.
- 강의에서는 출입구에서 도로까지 걸리는 시간을 0으로 가정한다.

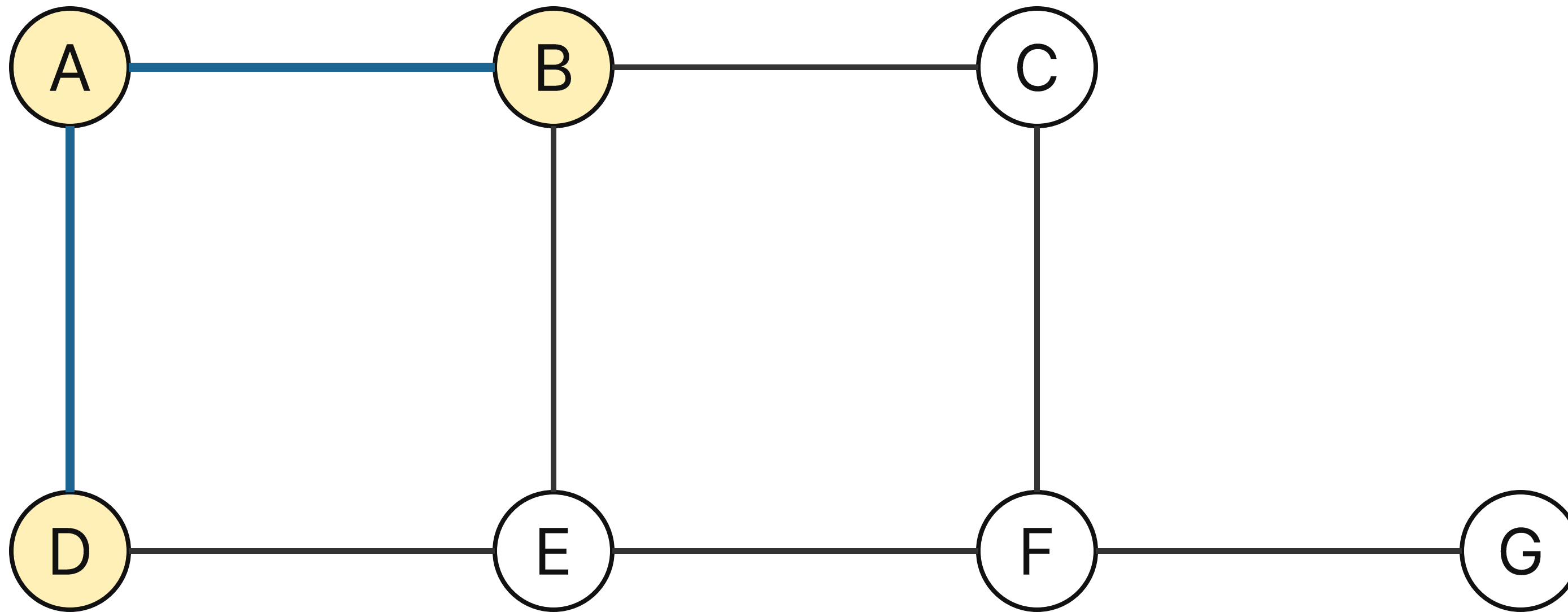
- 출발지에서 목적지까지 이동 경로가 있으면 연결되어 있다.
- 주택 A에서 직장 G까지는 갈 수 있지만 병원 H까지는 갈 수 없다.



- 그래프 탐색 Graph Traversal은 연결된 노드를 규칙에 따라 확인하는 과정이다.
 - 지도를 x 와 y 순서로 훑는 방식과 이동 가능한 연결을 따라가는 방식은 다르다.
- 출발지에서 도달할 수 있는 범위를 찾는다.
 - 목적지에 갈 수 있는지 / 어떤 순서로 갈 수 있는지 확인할 수 있다.

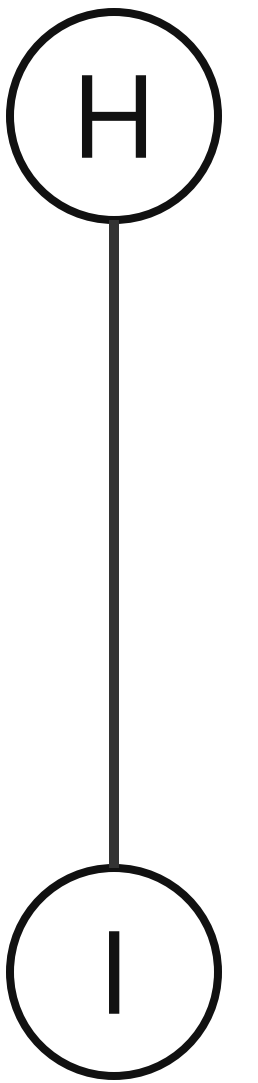
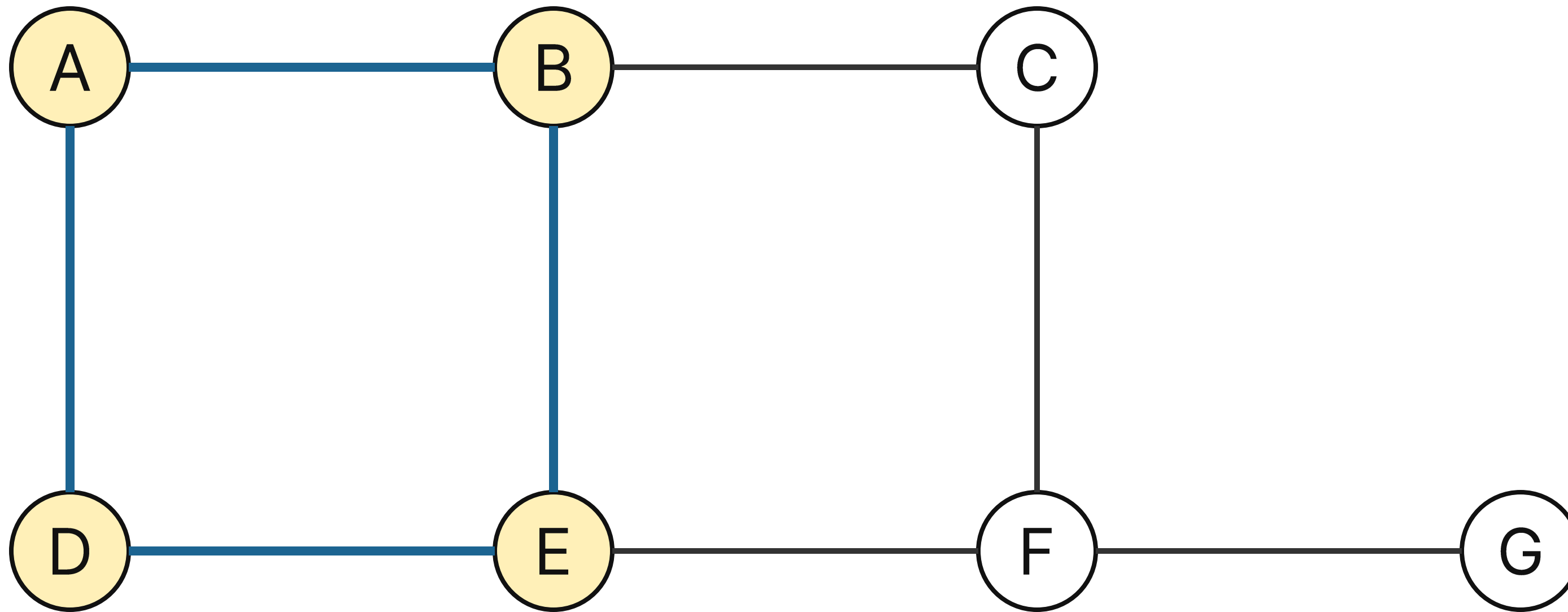
갈림길에서 남겨 둘 정보

- A에서 B와 D를 발견했다. B를 먼저 확인하면 D는 어떻게 기억할까?
- 아직 이웃을 확인하지 않은 노드를 별도의 목록에 남겨 둔다.



순환과 중복 탐색

- $A - B - E - D$ 를 따라가면 다시 A 로 돌아올 수 있다.
- 이미 발견한 노드를 기억하지 않으면 같은 도로를 계속 확인한다.



- *visited*는 이미 발견한 노드를 기록한다.
 - 새 노드를 대기 목록에 넣는 순간 함께 기록한다.
- 같은 노드를 발견해도 중복해서 목록에 넣지 않는다.
 - B에서 E를 발견했다면 D에서 E를 발견하더라도 다시 추가하지 않는다.
- *visited*에 있어도 아직 이웃을 확인하지 않았을 수 있다.

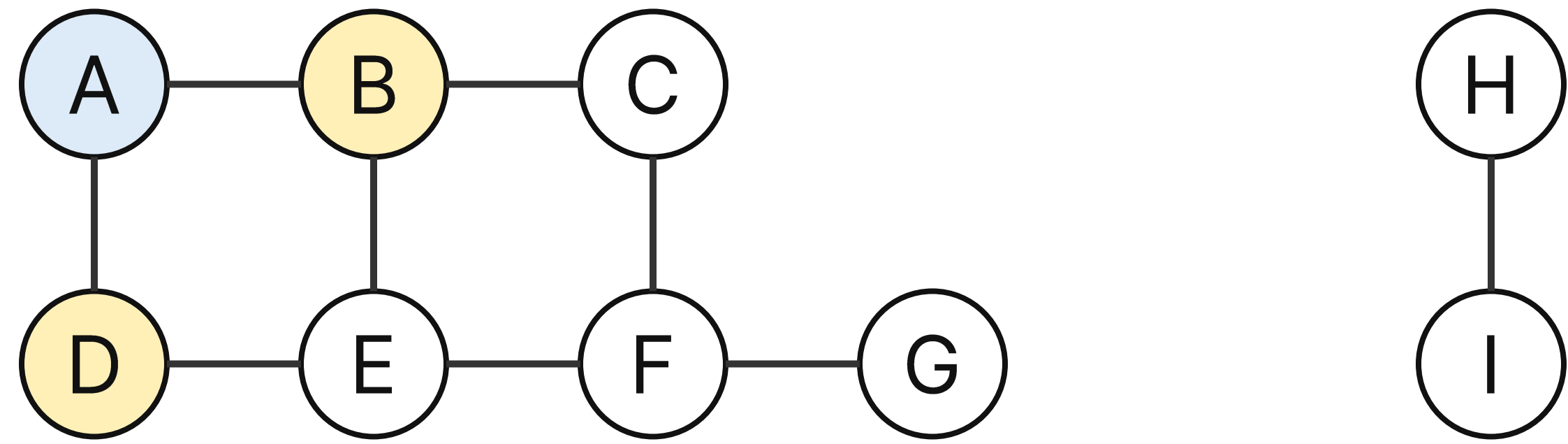
탐색 대기열 queue

- queue는 앞으로 이웃을 확인할 노드를 순서대로 보관한다.
- 먼저 들어온 노드를 먼저 꺼내는 FIFO 규칙을 사용한다.

동작	탐색 대기열 / 왼쪽이 앞
B와 D를 기다리게 한다	[B, D]
앞의 B를 꺼낸다	[D]
B의 새 이웃 C와 E를 뒤에 넣는다	[D, C, E]

발견한 노드와 확인한 노드

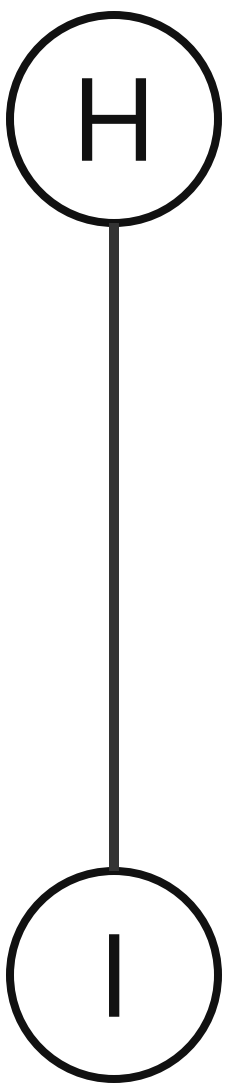
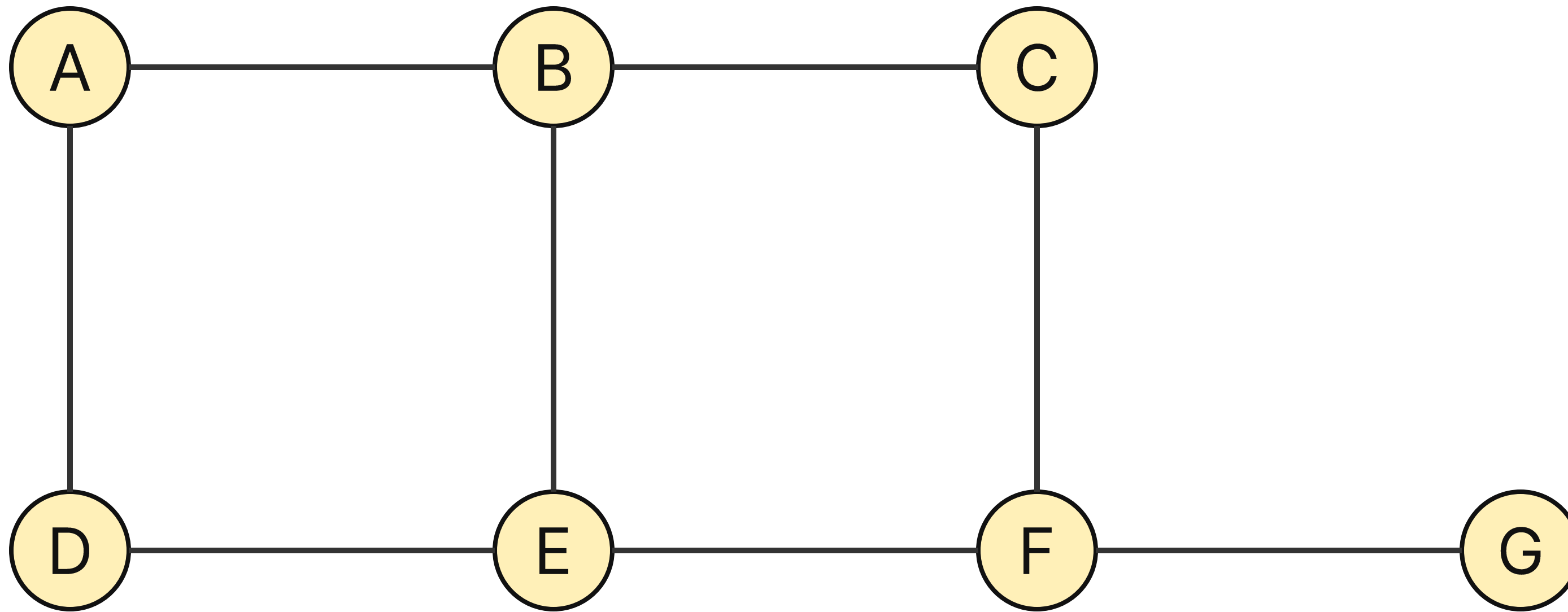
- A의 이웃을 확인한 직후에는 B와 D가 처리 차례를 기다린다.
- 파란색 : 이웃 확인 완료, 노란색 : 확인 대기, 흰색 : 아직 발견하지 못함



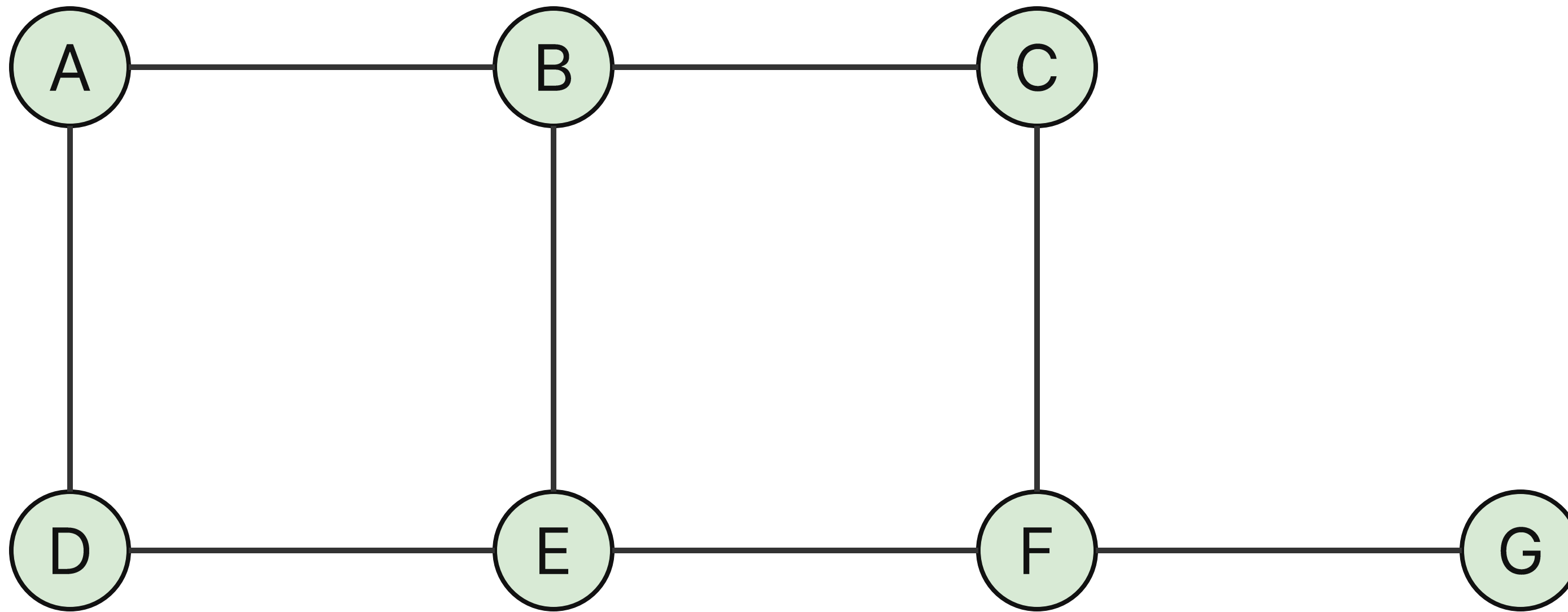
자료	내용
visited	{A, B, D}
queue	[B, D]
확장 완료	A

탐색으로 확인한 연결 범위

- A에서 시작하면 A부터 G까지 발견한다.
- 탐색 대기열이 비어도 H와 I는 방문하지 못한다.
- A에서 도달 가능한 노드 : {A, B, C, D, E, F, G}



- 연결 요소 Connected Component는 서로 경로로 연결된 노드의 묶음이다.
 - 연결된 노드를 더 추가할 수 없는 범위까지 포함한다.
 - 연결 요소 1 : {A, B, C, D, E, F, G} / 연결 요소 2 : {H, I}



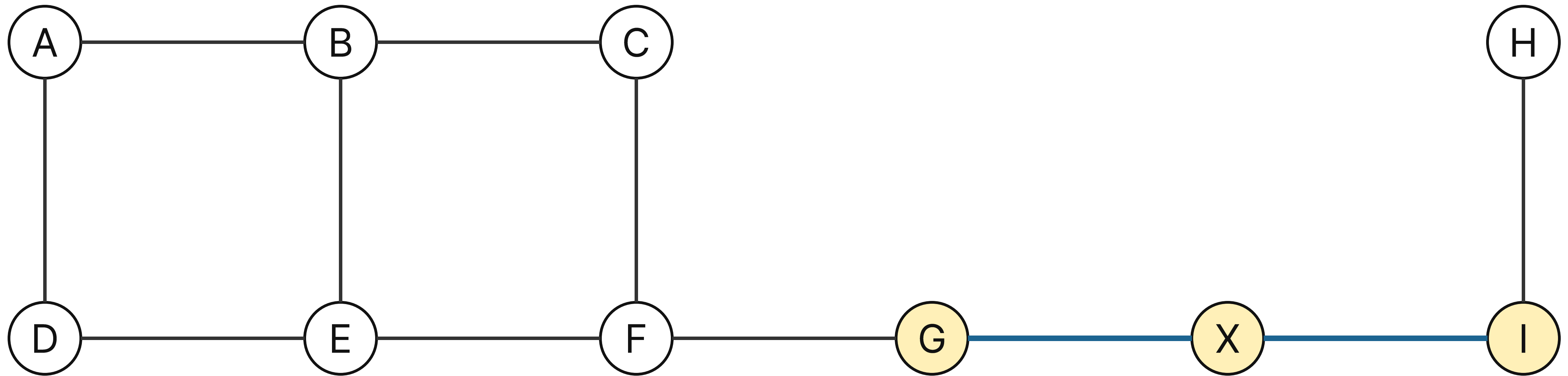
연결 요소의 번호 매기기

- 아직 번호가 없는 노드에서 탐색을 시작한다.
- 한 번의 탐색으로 발견한 노드에 같은 번호를 부여한다.

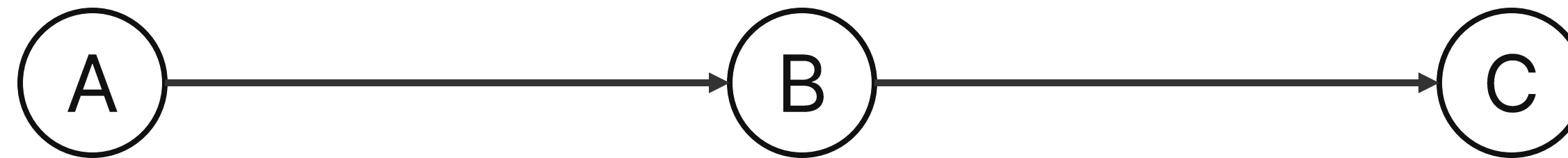
순서	탐색 시작	같은 번호를 받는 노드
1	A	A / B / C / D / E / F / G
2	H	H / I
종료	미분류 노드 없음	연결 요소 2개

도로 하나를 추가한 결과

- 빈 셀 (4, 2)에 도로 X를 추가한다.
- $G - X$ 와 $X - I$ 가 생기면서 두 연결 요소가 하나로 합쳐진다.
- 노드 10개 / 무방향 간선 11개 / 연결 요소 1개

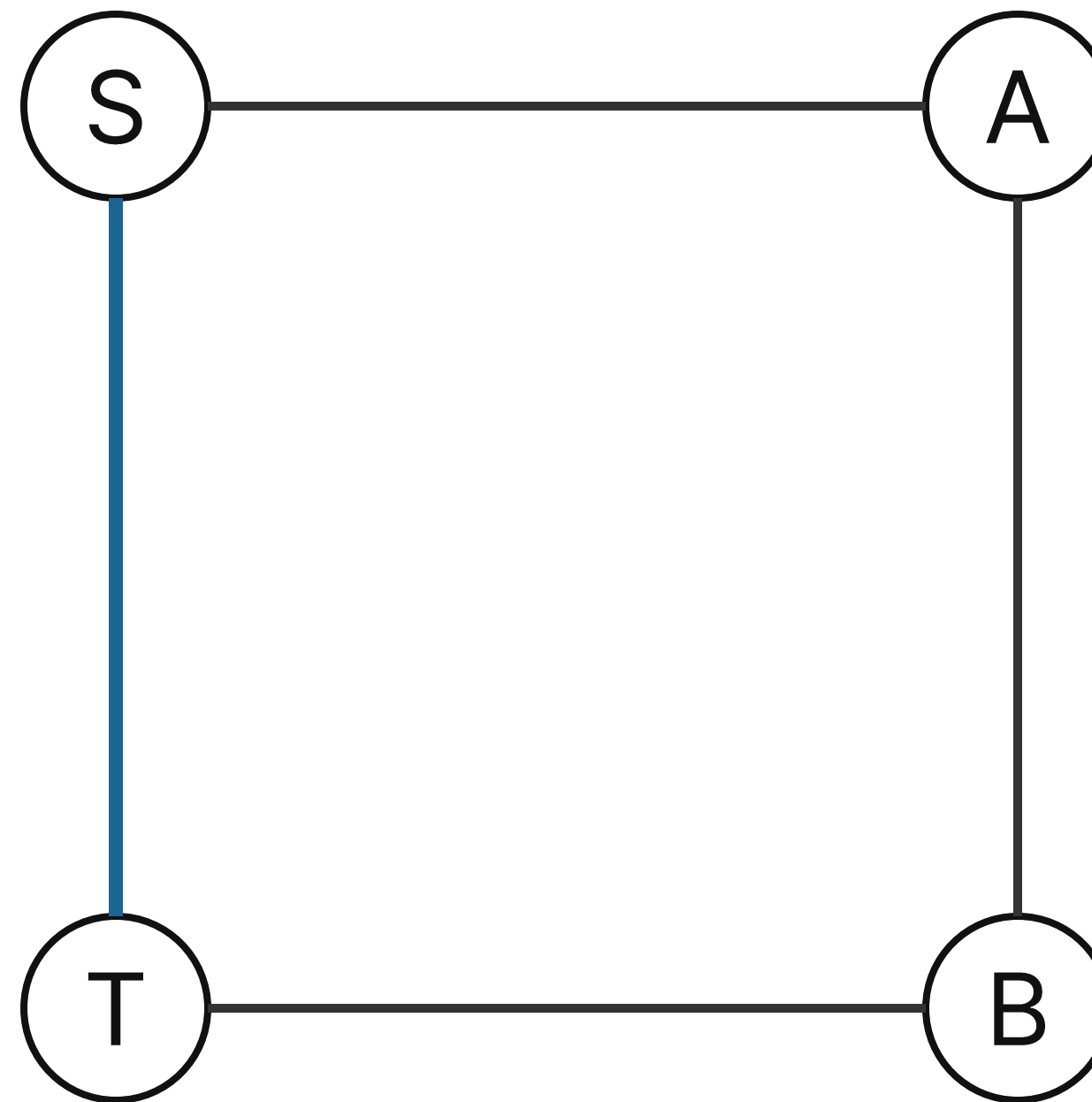


- 방향이 생기면 출발지와 목적지의 순서가 중요하다.
- A에서 C로 갈 수 있어도 C에서 A로 돌아오지 못할 수 있다.



BFS와 DFS의 탐색 순서

- 너비 우선 탐색 Breadth-First Search; BFS은 가까운 단계부터 확인한다.
- 깊이 우선 탐색 Depth-First Search; DFS은 한 방향으로 깊게 탐색한다.



BFS의 탐색 순서

- BFS는 가까운 단계부터 확인한다.
 - 0개 간선 / 1개 간선 / 2개 간선 순서로 범위를 넓힌다.

간선 수	이번 단계에서 처음 발견한 노드
0	A
1	B / D
2	C / E
3	F
4	G

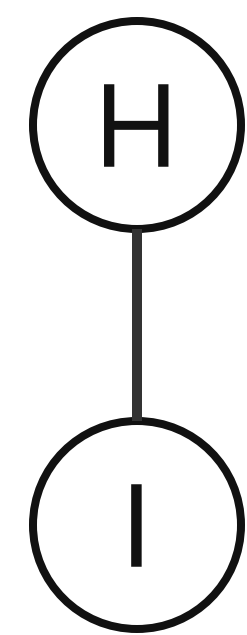
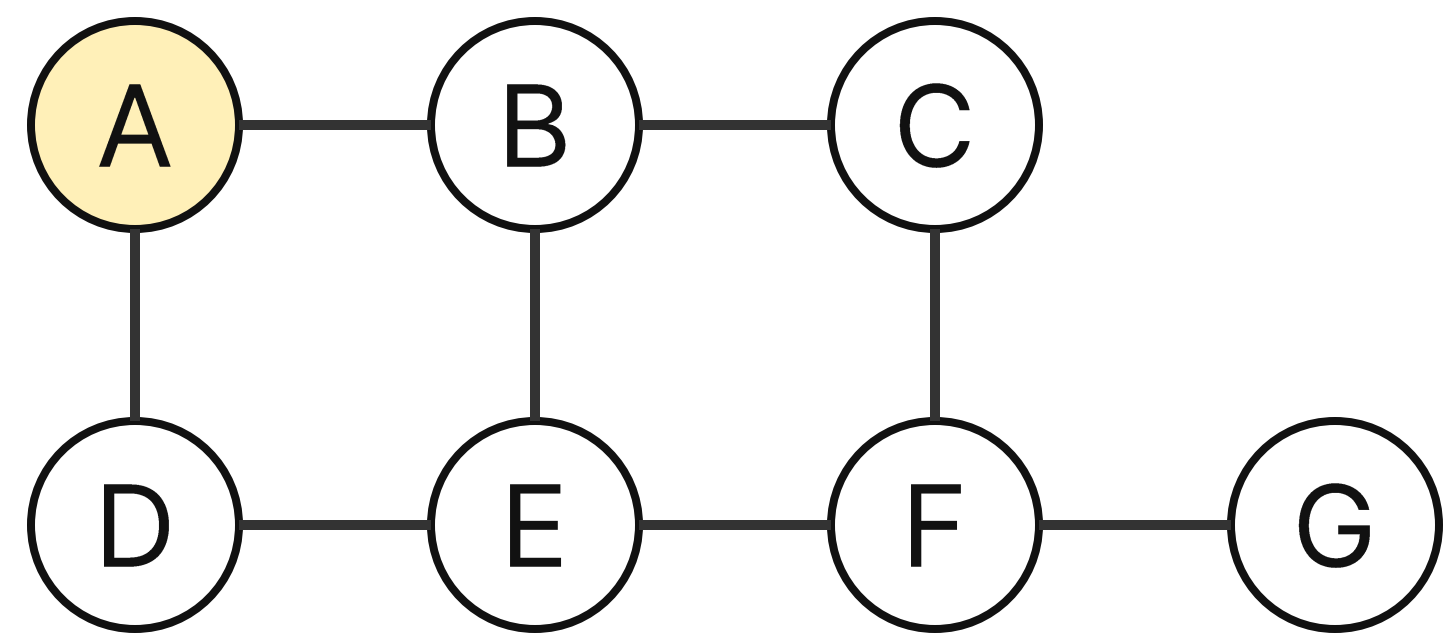
BFS가 저장하는 상태

- 탐색 대기열과 방문 기록에 거리와 이전 노드를 추가한다.

자료	역할	초기 상태
queue	앞으로 확장할 노드	[A]
visited	이미 발견한 노드	{A}
dist	출발지부터의 간선 수	A : 0
parent	처음 발견한 이전 노드	A : 없음

BFS 예제의 초기 상태

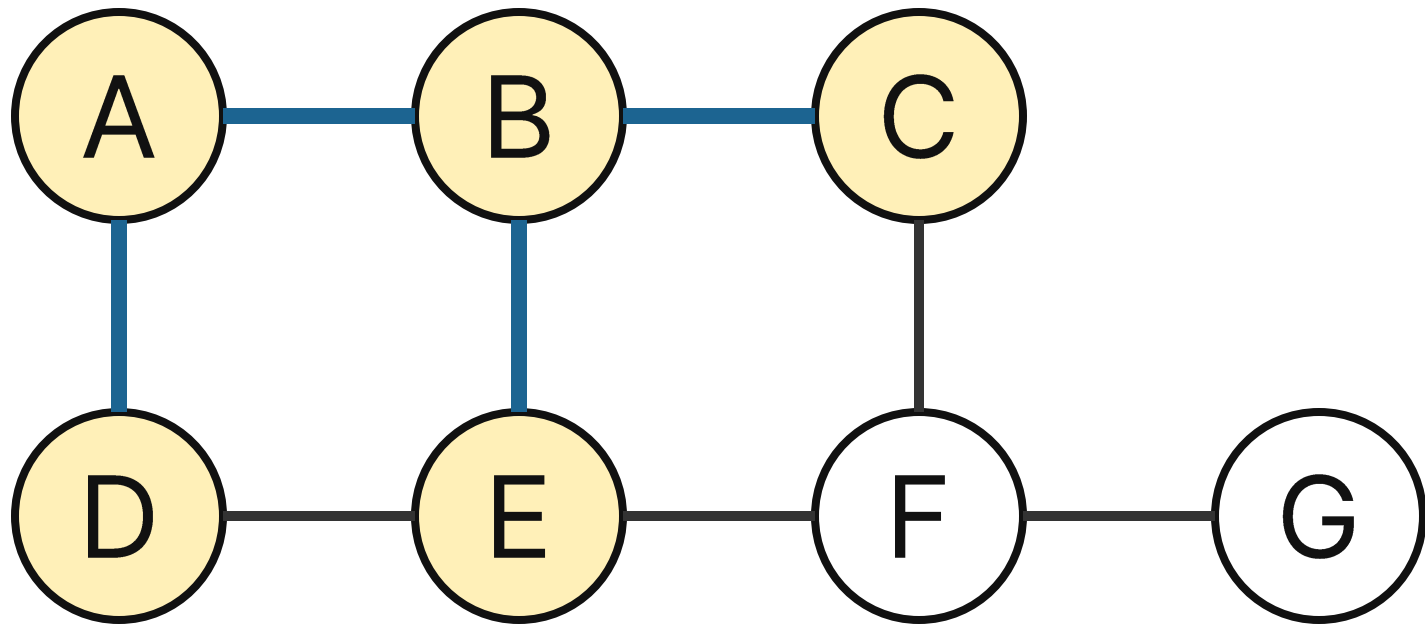
- A에서 시작하며 이웃은 오른쪽 / 아래 / 왼쪽 / 위 순서로 본다.



자료	값
queue	[A]
dist	A : 0
parent	A : 없음

BFS 예제 : A와 B 확장

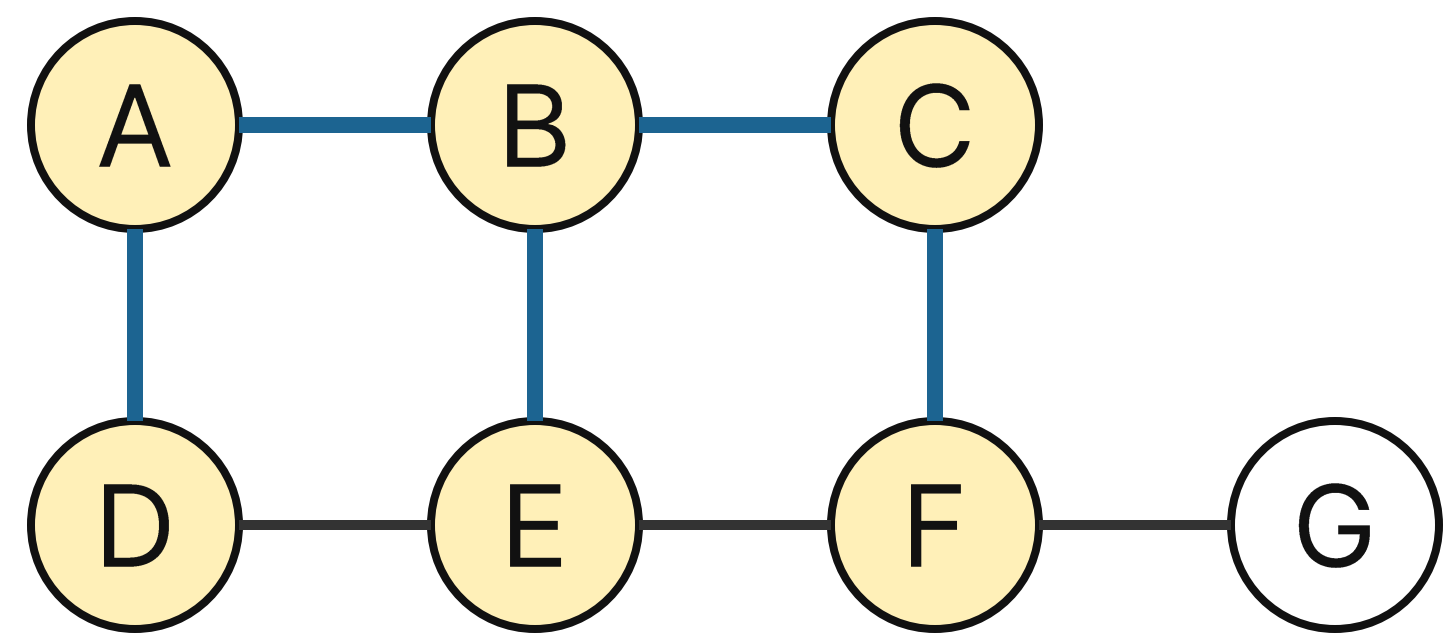
- A를 꺼내 B와 D를 넣고, B를 꺼내 C와 E를 넣는다.



처리 후	queue
A 확장	[B, D]
B 확장	[D, C, E]
새 거리	B/D : 1 C/E : 2

BFS 예제 : D와 C 확장

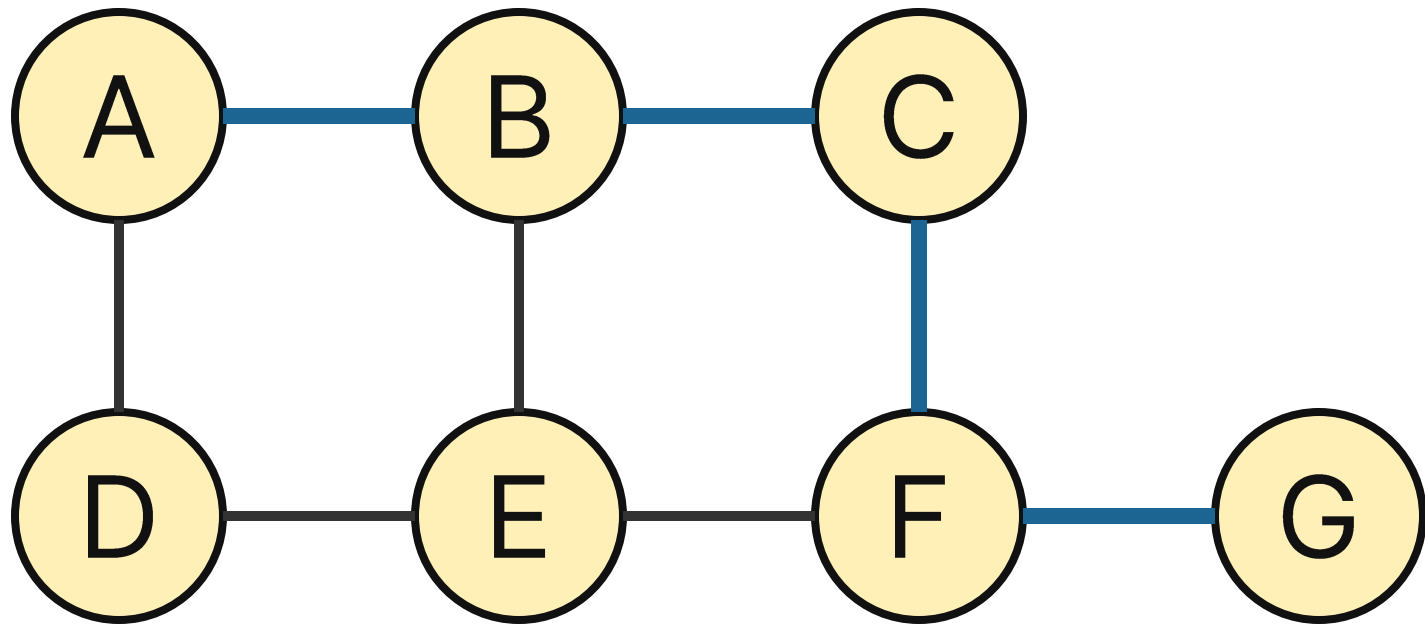
- D에서는 새 노드가 없고, C에서 F를 처음 발견한다.



처리 후	queue
D 확장	[C, E]
C 확장	[E, F]
새 기록	dist[F]=3 parent[F]=C

BFS 예제 : E와 F 확장

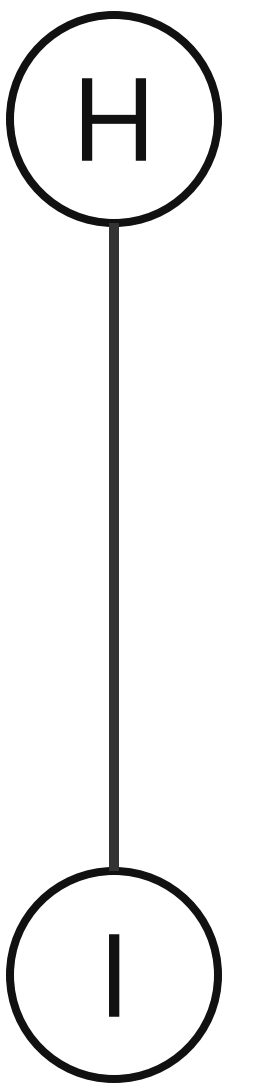
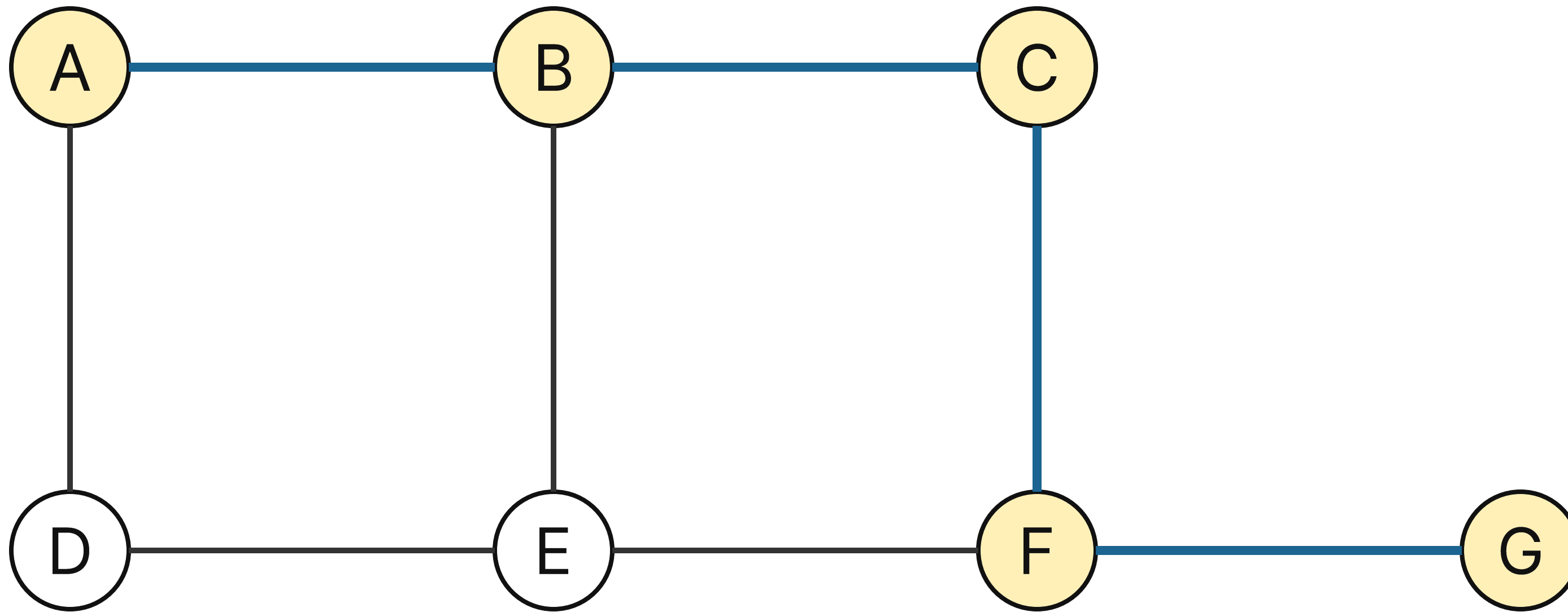
- E에서는 새 노드가 없고, F에서 목적지 G를 발견한다.



처리 후	queue
E 확장	[F]
F 확장	[G]
새 기록	dist[G]=4 parent[G]=F

이전 노드로 경로 복원

- 목적지부터 parent를 따라 출발지까지 거슬러 간다.
- G / F / C / B / A를 뒤집으면 실제 이동 순서가 된다.



- 최단 경로 Shortest Path는 정해진 비용의 합이 가장 작은 경로다.
 - 간선 비용이 모두 같으면 간선 수가 적은 경로를 찾는다.
- 기본 도시에서 A부터 G까지의 경로를 찾는다.
 - 이번 계산은 X를 추가하기 전의 도로망을 사용한다.
- 경로를 찾기 전에 출발지 / 목적지 / 비용을 정한다.

- 최단 경로는 여러 개일 수 있다.
 - $A - B - C - F - G$ 와 $A - D - E - F - G$ 는 모두 간선 4개다.
- 출발지와 목적지가 같으면 이동 간선은 0개다.
 - A에서 A로의 경로는 [A]로 표현할 수 있다.
- 도달할 수 없으면 경로 없음으로 구분한다.
 - 기본 도시의 A에서 H까지는 거리를 0으로 기록하지 않는다.

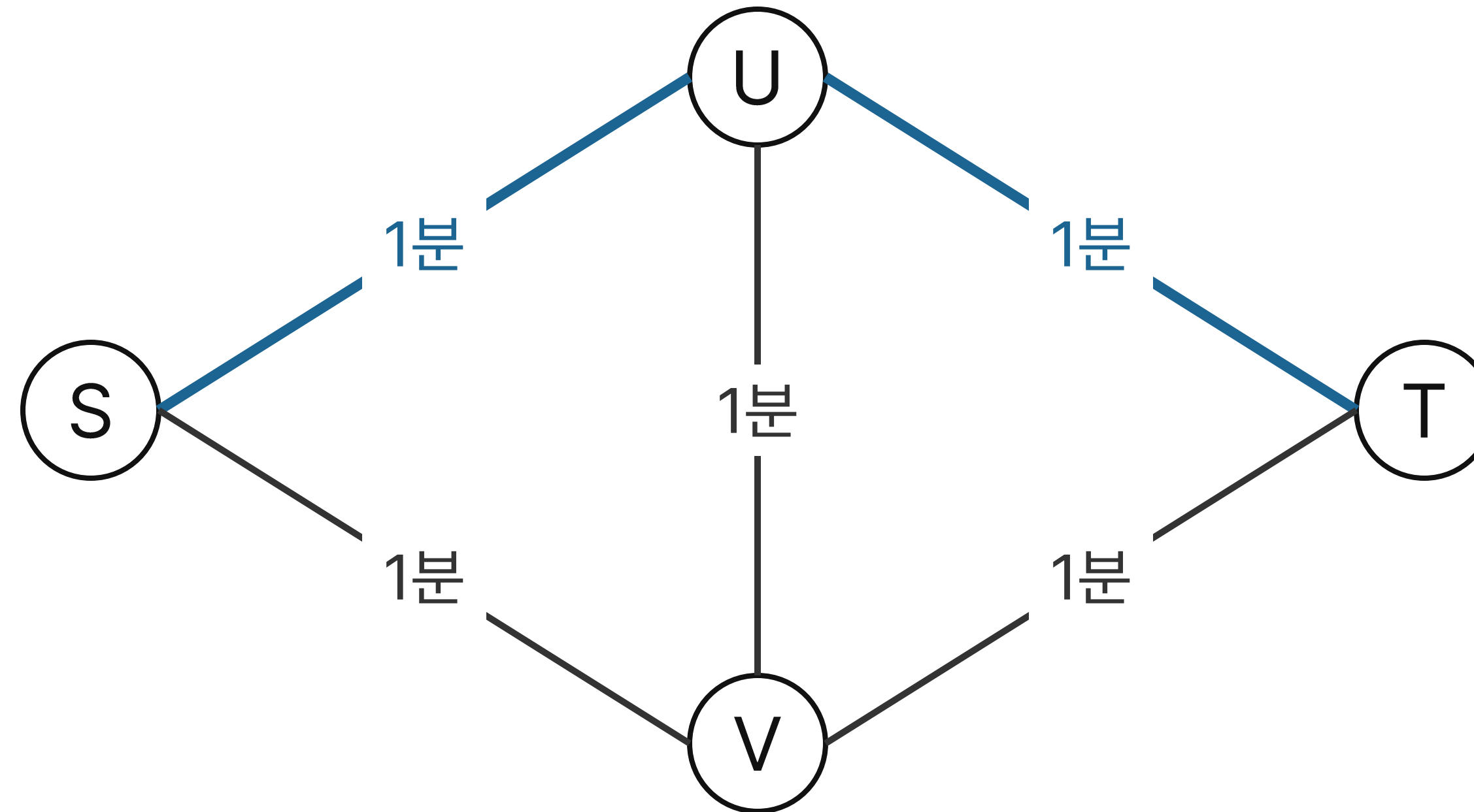
- 방문 표시가 없으면 같은 도로를 반복해서 탐색한다.
- BFS에서 parent를 나중에 덮어쓰면 경로 복원이 꼬일 수 있다.
- 대각선 도로를 연결하면 실제 규칙에 없는 지름길이 생긴다.
- 화면의 선 교차를 모두 교차로로 처리하면 연결이 달라진다.

BFS가 최소화하는 비용

- 앞에서 구한 BFS 경로는 지나가는 간선 수가 가장 적다.
 - 모든 구간이 100m라면 거리 / 모두 1분이라면 시간도 최소다.
- 이 성질은 모든 간선의 비용이 같을 때 성립한다.
- 구간별 시간이 달라져도 BFS로 가장 빠른 길을 찾을 수 있을까?

시간이 같은 도로에서의 BFS

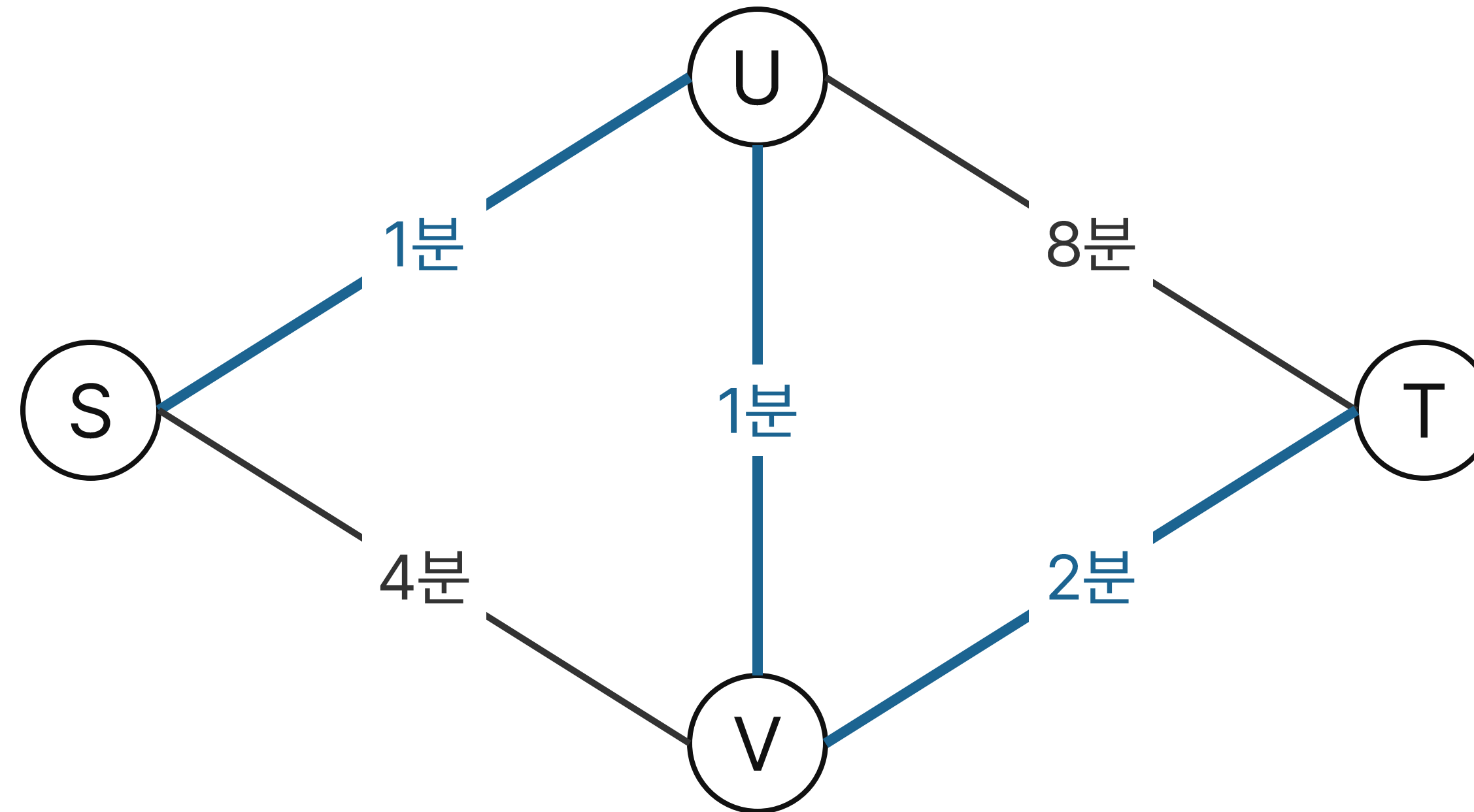
- BFS의 조건을 확인하기 위해 S에서 T까지의 시간을 비교한다.
- 먼저 모든 간선을 1분으로 두면 2간선 경로가 2분으로 가장 빠르다.



이제 도로 연결은 유지하고, 구간별 이동시간만 바꾸어 보자.

구간별 이동시간이 달라진 도로

- 바로 앞 도로의 연결과 길이는 유지하고, 구간별 시간만 바꿨다.
- 간선이 적은 $S - U - T$ 는 9분 / 간선이 더 많은 $S - U - V - T$ 는 4분이다.



이 도로에서도 BFS가 가장 빠른 4분 경로를 찾을 수 있을까?

BFS 결과와 최소 시간 경로

- BFS는 간선 수를 비교하므로 간선이 2개인 경로를 먼저 찾는다.
- 그래서 6분 또는 9분 경로를 선택하고, 간선이 3개인 4분 경로를 놓친다.

경로	간선 수	이동시간
$S - U - T$	2	$1 + 8 = 9\text{분}$
$S - V - T$	2	$4 + 2 = 6\text{분}$
$S - U - V - T$	3	$1 + 1 + 2 = 4\text{분}$

가장 빠른 경로를 찾으려면 다음 후보도 누적시간을 기준으로 골라야 한다.

큐에서 후보를 꺼내는 기준

- 다음 후보는 등록 순서가 아니라 누적시간 순서로 고른다.
- S에서 V(4분), U(1분) 순서로 등록해도 U를 먼저 꺼낸다.

방법	먼저 꺼낼 후보	선택 이유
BFS의 FIFO 큐	V	먼저 등록한 후보
누적시간 우선순위 큐	U	현재 비용이 1분으로 작음

이제 도로 연결은 유지하고, 구간별 이동시간만 바꾸어 보자.

BFS에서 바뀌는 두 규칙

- U를 거쳐 비용이 줄어들면, 이미 발견한 V의 기록도 갱신한다.
- 후보 선택 기준과 비용 갱신 규칙을 함께 바꾼다.

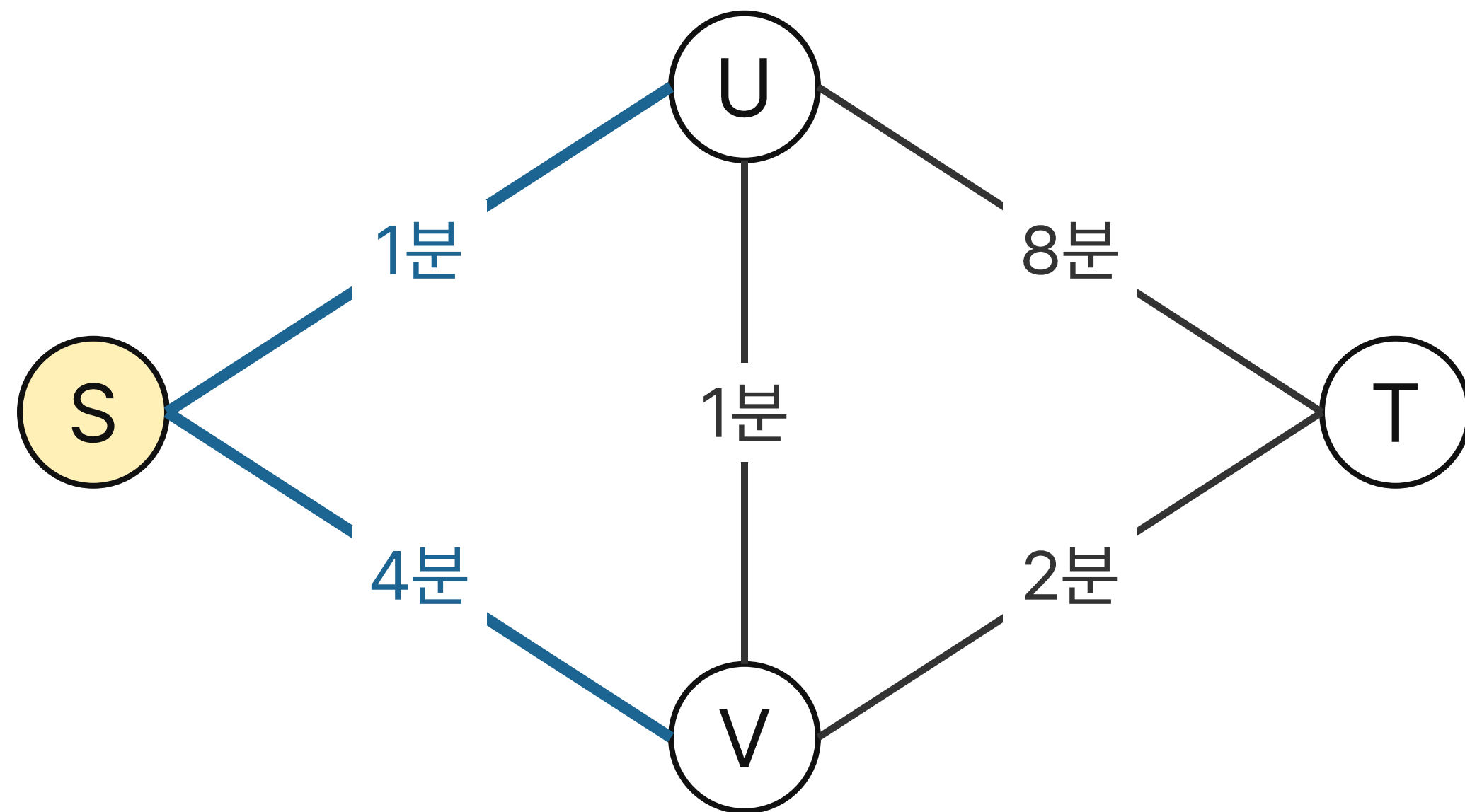
비교 항목	BFS	시간 비용을 비교하는 규칙
후보 선택 기준	먼저 등록한 노드	누적시간이 작은 노드
비용 갱신 규칙	처음 발견할 때 기록	더 작은 시간이 나오면 갱신
이전 노드	첫 발견의 parent 유지	시간이 줄면 parent도 변경

이 두 규칙을 함께 적용하는 방법이 다익스트라 알고리즘이다.

- 다익스트라 알고리즘 Dijkstra's Algorithm은 비용이 최소가 되도록 후보를 선택하고 더 작은 비용으로 갱신하는 작업을 반복 수행한다.
- 간선 비용이 0 이상이면 선택한 노드의 최소 비용을 확정할 수 있다.
- 후보 비용 = $dist[\text{현재 노드}] + \text{간선 비용}$
- 새 비용이 더 작으면 $dist$ 와 $parent$ 를 함께 바꾼다.
- 이제 S / U / V / T 도로망을 출발지부터 순서대로 계산한다.

다익스트라 : 시작점 S

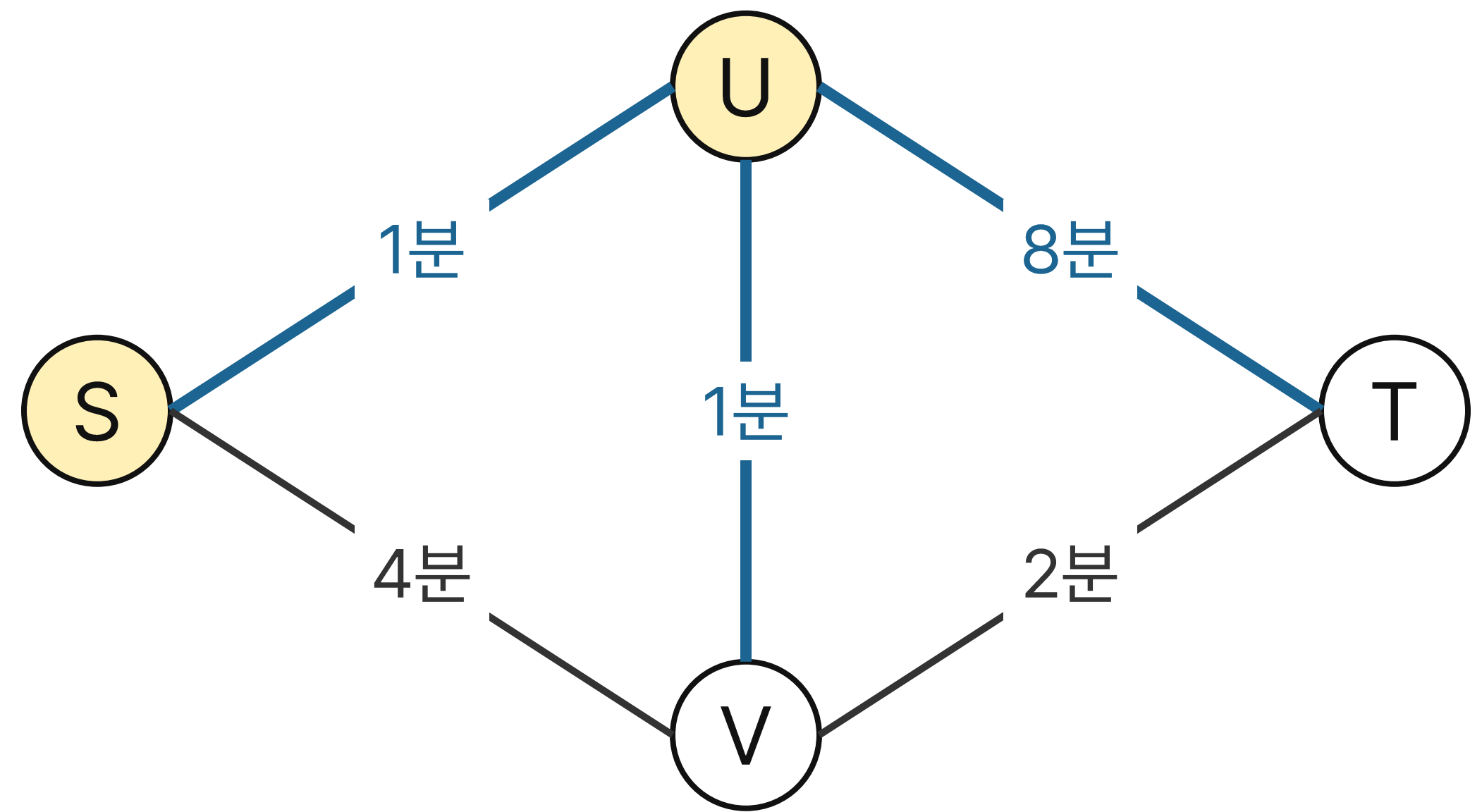
- 먼저 S의 0분을 확정하고 U에 1분, V에 4분을 기록한다.
- 후보 중 U의 1분이 가장 작으므로 다음에는 U를 선택한다.



노드	현재 비용	확정
S	0	예
U	1	아니요
V	4	아니요
T	∞	아니요

다익스트라 : U 선택

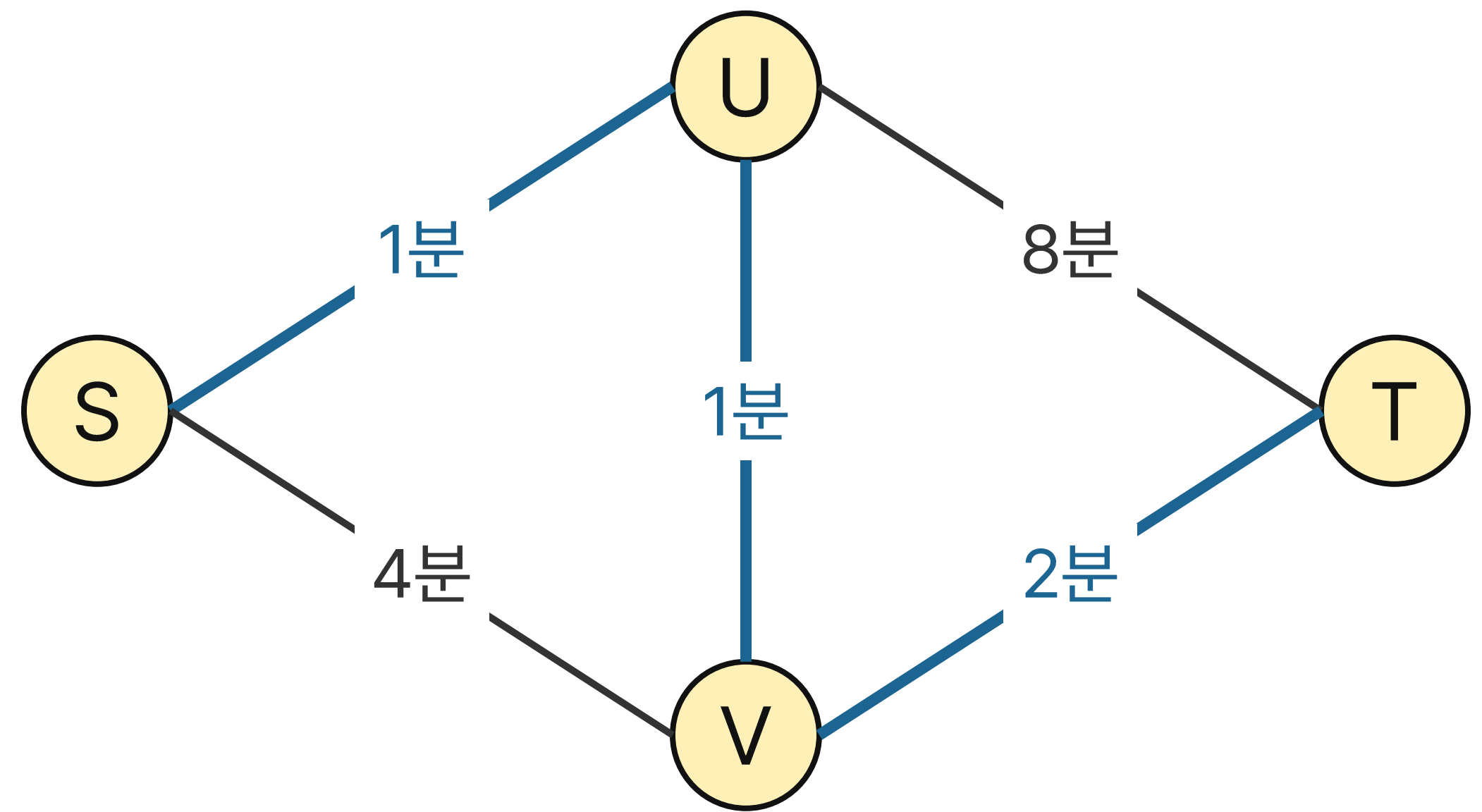
- U를 선택해 V를 2분으로 갱신하고, T에는 후보 비용 9분을 기록한다.
- 미확정 후보 V(2분) / T(9분) 중 V를 다음에 선택한다.



노드	현재 비용	이전 노드
S	0 / 확정	없음
U	1 / 확정	S
V	2	U
T	9	U

다익스트라 : V와 T 선택

- 앞에서 고른 V를 거치면 T의 비용은 9분에서 4분으로 줄어든다.
- T의 4분이 가장 작은 미확정 비용이 되어 마지막으로 T를 선택한다.

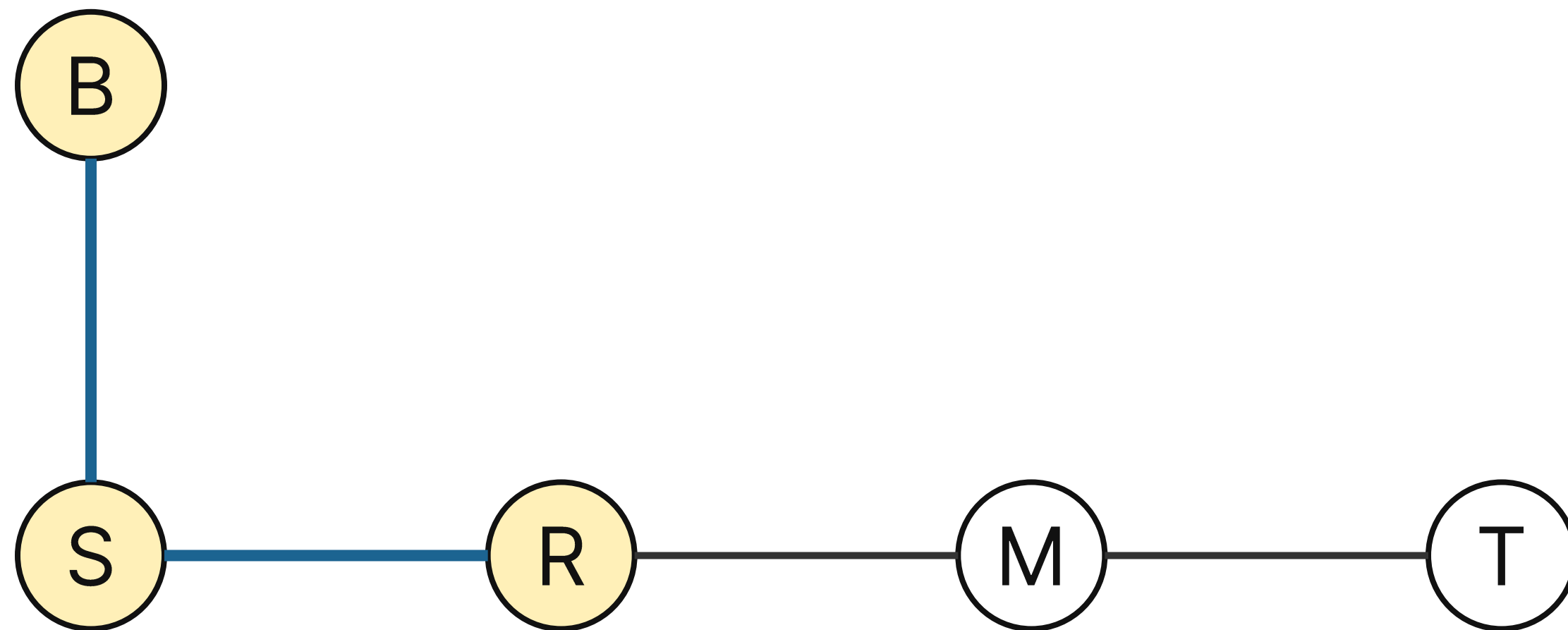


노드	최소 시간	이전 노드
S	0	없음
U	1	S
V	2	U
T	4	V

- 같은 도로에서 다익스트라는 4분 경로를 찾았다.
 - BFS의 2간선 경로는 6분 또는 9분 / $S - U - V - T$ 는 4분이다.
- T를 처음 발견한 9분에 종료하면 최소 비용을 놓치게 된다.
 - 미확정 후보 중 최소 비용으로 선택되는 4분에 종료해야 한다.
- 하지만 최단 경로를 찾는 과정에서 확인하는 후보가 너무 많다.
같은 최단 경로를 찾으면서 확인할 후보를 줄일 수 있을까?

최단 경로와 탐색 범위

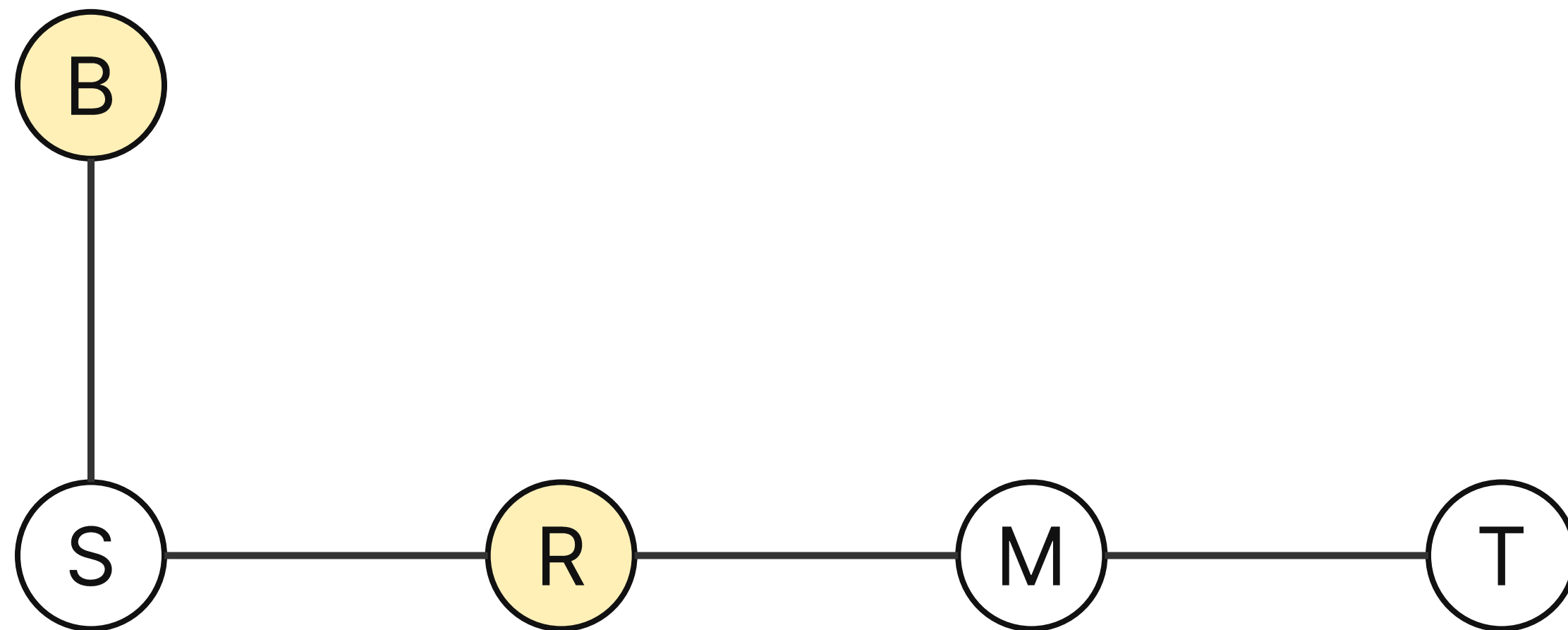
- 다익스트라는 누적비용만 비교하므로 다른 방향의 B도 확인한다.
- 다음은 탐색 범위를 비교하기 위한 예제이며, 모든 간선의 이동시간은 1분이다.
- 관점을 바꿔 목적지까지 남은 비용을 비교해보는 건 어떨까?



후보	누적시간 g(분)
B	1
R	1

남은 시간을 예상하는 방법

- 목적지 $T = (3, 1)$ 까지의 예상 남은 시간을 h 라고 쓴다.
- 4방향 / 모든 간선 1분 : $h = |\text{현재 } x - 3| + |\text{현재 } y - 1|$
- 예상 남은 시간은 B의 경우 4분, R의 경우 2분이다.



후보 좌표	h 계산	$h(\text{분})$
B (0,0)	$3 + 1$	4
R (1,1)	$2 + 0$	2

- A* 알고리즘 A* Algorithm은 누적비용과 남은 비용의 추정값을 함께 사용해 탐색 순서를 정한다.
- $f(n) = g(n) + h(n)$
 - f 가 작은 후보부터 선택한다.
 - g : 현재까지 찾은 최소 누적비용
 - h : 목적지까지의 추정 비용
- h 를 계산하는 함수를 휴리스틱이라고 한다.
 - f 는 예상 총비용이다.
 - 최종 경로의 실제 비용은 목적지의 g 다.

A*의 후보 선택 기준

- 앞에서 구한 g 와 h 를 더하면 B는 5분 / R은 3분이다.
 - B : $1 + 4 = 5$
 - R : $1 + 2 = 3$
- A*는 f 가 더 작은 R을 먼저 선택한다.
 - h 만 비교하지 않고, 이미 사용한 비용 g 도 함께 반영한다.
- 모든 노드에서 $h = 0$ 이면 다익스트라 알고리즘과 같은 기준이 된다.

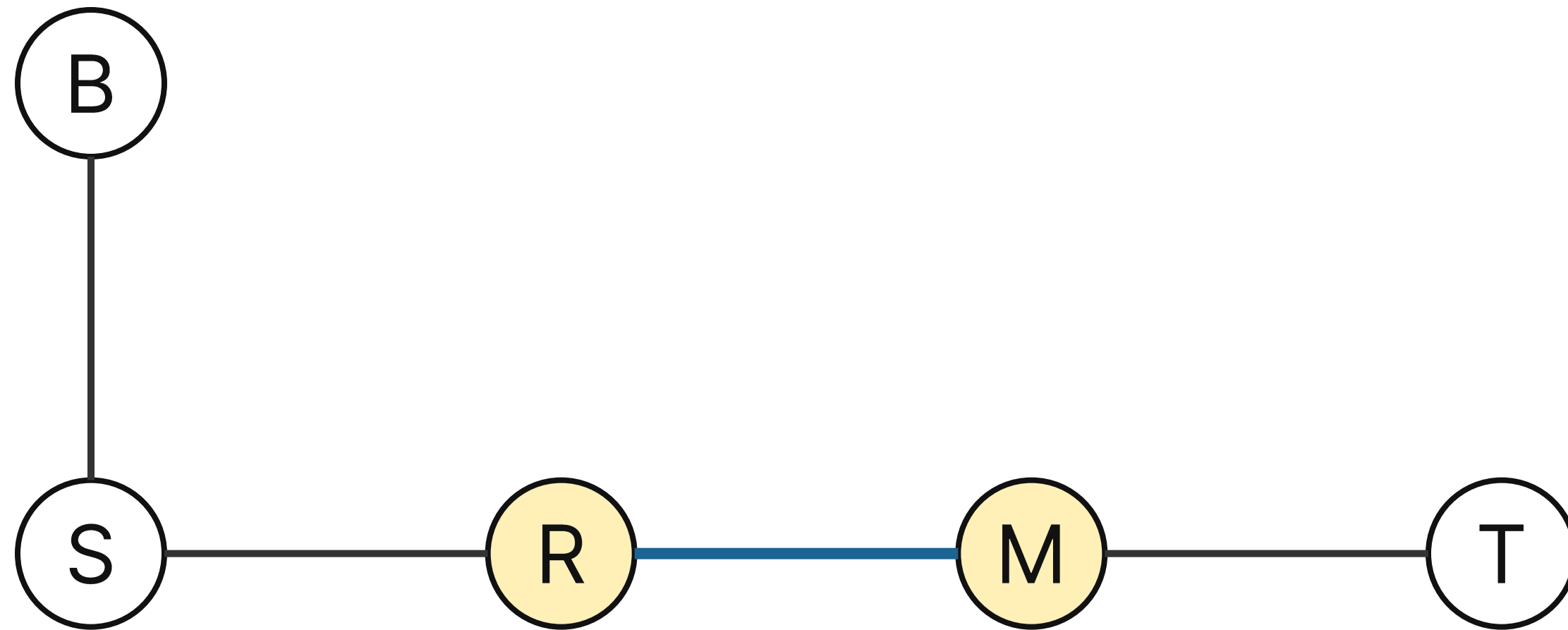
- 후보 선택은 f 로, 더 좋은 경로인지의 판단은 g 로 한다.
- 후보는 앞으로 확인할 노드 / parent는 경로 복원에 쓸 이전 노드다.

단계	수행할 동작
1. 초기화	$g(S) = 0$ / 나머지 $g = \infty$ S 를 $f(S) = h(S)$ 로 후보에 등록
2. 후보 선택	f 가 가장 작은 후보를 꺼낸다. 목적지라면 경로를 복원하고 종료한다.
3. 이웃 갱신	새로운 g 가 더 작으면 g 와 parent를 바꾼다. 이웃을 후보에 등록하거나 f 를 갱신한다.
4. 반복	후보가 남으면 2단계로 돌아간다. 목적지를 못 찾고 후보가 없으면 도달 불가

- 선택한 노드 u 를 거쳐 이웃 v 에 가는 비용을 계산한다.
 - 새 비용 = $g(u) + \text{간선 비용}(u, v)$
- 새 비용이 기존 $g(v)$ 보다 작을 때만 기록을 바꾼다.
 - $g(v) = \text{새 비용}$
 - $\text{parent}(v) = u$
- 후보의 우선순위도 $f(v) = g(v) + h(v)$ 로 갱신한다.
 - 처음 발견한 노드는 $g = \infty$ 이므로 유한한 경로를 찾으면 갱신한다.

A* 예제 : R에서 M으로

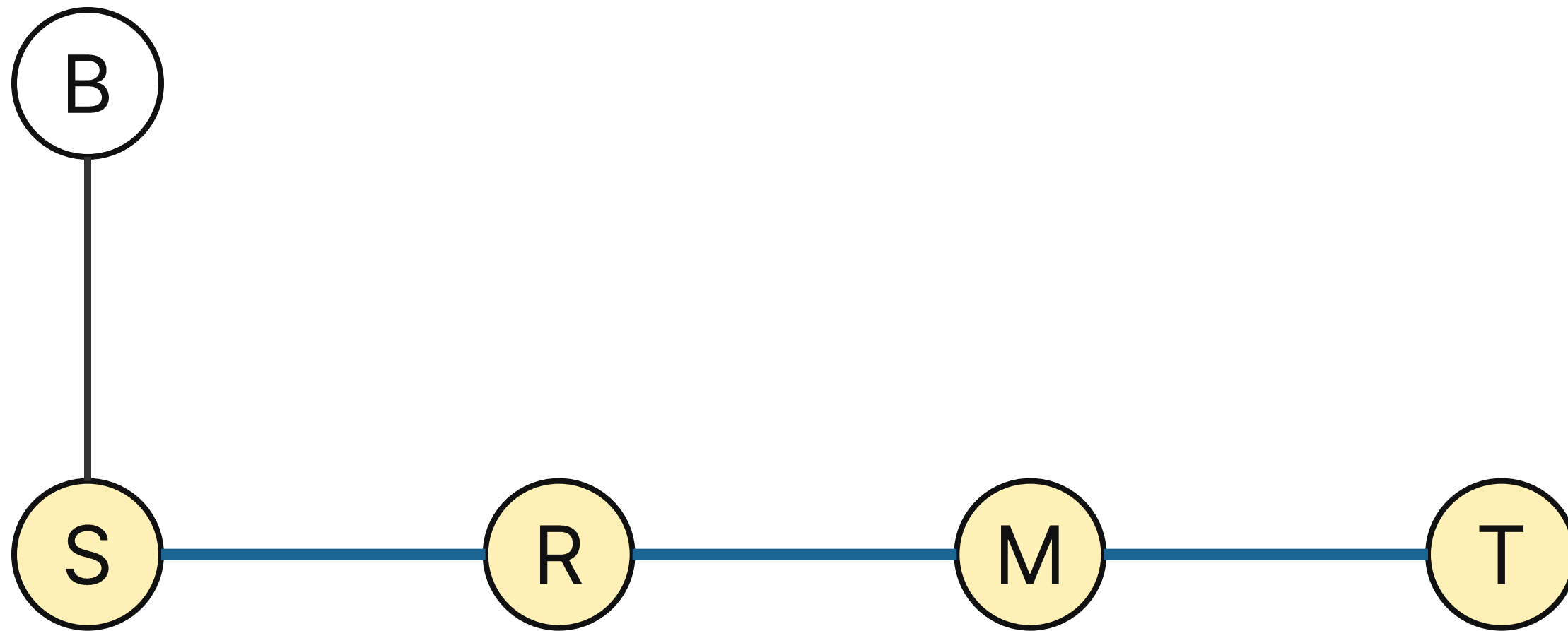
- R을 거쳐 M에 가면 $g(M) = 1 + 1 = 2$ 분이다.
- $g(M) : \infty \rightarrow 2$ / $parent(M) = R$ / $h(M) = 1$ / $f(M) = 3$
- 갱신 후 후보는 B와 M, f 가 더 작은 M을 다음에 선택한다.



후보	$g(\text{분})$	$h(\text{분})$	$f(\text{분})$
B	1	4	5
M	2	1	3

A* 예제 : 목적지 T 선택

- M을 거쳐 T에 가면 $g(T) = 3\text{분}$ / $h(T) = 0$ / $f(T) = 3$ 이다.
- $\text{parent}(T) = M$ / T를 후보에 넣고, 최소 f 로 꺼낼 때 종료한다.
- parent 로 복원한 경로는 $S \rightarrow R \rightarrow M \rightarrow T$ / 실제 이동시간은 $g(T) = 3\text{분}$ 이다.



후보	$g(\text{분})$	$h(\text{분})$	$f(\text{분})$
B	1	4	5
T	3	0	3

같은 지도에서의 탐색 순서

- 방금 실행한 A*와 다익스트라는 모두 같은 3분 경로를 찾는다.
- 다익스트라에서 g 가 같은 B와 R은 B부터 선택한다고 정한다.

순서	다익스트라	$g(\text{분})$	A*	$f(\text{분})$
1	S	0	S	3
2	B	1	R	3
3	R	1	M	3
4	M	2	T	3
5	T	3	종료	—

- A*가 최소 시간을 보장하려면 h 를 실제 남은 최소 비용 이하로 둔다.
- 한 번 확인한 노드를 다시 열지 않는 구현에는 일관된 h 가 필요하다.
 - $h(u) \leq \text{간선 비용}(u, v) + h(v)$
 - $h(\text{목적지}) = 0$
- 시간 비용을 추정할 때는 거리 값을 시간 단위로 바꾼다.
 - 예 : 실제 위치의 직선거리 / 전체 도로의 가능한 최대 속도

다른 최단 경로 문제

- 지금까지는 비용이 음수가 아닌 출발지가 한 곳인 문제를 다뤘다.
- 비용이 음수이거나 출발지가 여러 곳인 경우의 결과가 필요하면 문제의 조건이 달라진다.

문제	예시 방법	이번 도로 모형과의 관계
음수 간선 포함	Bellman-Ford	이동시간은 보통 비음수
모든 노드 쌍	Floyd-Warshall	큰 도로망에서는 비용이 큼
한 출발지 / 여러 목적지	Dijkstra	여러 목적지의 시간을 한 번에 계산

최단 경로 알고리즘의 선택

- 도로의 비용과 목적지 조건에 맞춰 알고리즘을 선택한다.

방법	다음 후보의 선택 기준	적용 조건
BFS	먼저 등록한 노드	모든 간선 비용이 같음
Dijkstra	현재 비용 g 가 최소	비용이 음수가 아님
A^*	현재 비용 g + 예상 비용 h	특정 목적지 / 적절한 h

- 경로가 정해진 뒤에는 그 경로를 누가 언제 이용하는지 정해야 한다.
 - 혼잡을 계산하려면 누가 언제 얼마나 이동하는지도 필요하다.
- 출발지와 목적지별 이동 수요를 만든다.
 - 예 : 주택 A에서 직장 G로 1분 동안 차량 8대가 출발한다.
- 같은 경로에 많은 이동이 겹치면 병목이 생길 수 있다.

- 이동하려는 차량 수와 실제로 통과한 차량 수를 구분한다.
 - 1분에 8대가 도착해도 5대만 통과할 수 있다.
- 처리 용량은 일정 시간 동안 처리할 수 있는 최대 대수다.
 - 용량이 5대/분이면 1분 구간의 처리 가능 대수는 5대다.
- 도착한 차량을 모두 처리하지 못하면 대기 차량이 쌓인다.

- 각 1분 구간에서 대기 차량과 새 도착 차량을 합친다.
- 예제에서는 새로 도착한 차량도 같은 구간에서 처리한다.

예제 : 4분 동안의 대기열

- 처음 대기는 0대, 용량은 5대/분으로 고정한다.
- 도착량이 용량보다 작아진 뒤에도 대기열은 한동안 남는다.

구간	시작 대기	도착	통과	종료 대기
0~1분	0	8	5	3
1~2분	3	8	5	6
2~3분	6	2	5	3
3~4분	3	2	5	0

이동 중에 바뀌는 교통 상황

가상공간과 메타버스
Road Networks and Traffic

- 경로를 선택한 뒤에도 이동시간은 달라질 수 있다.
 - 많은 차량이 같은 우회로를 선택하면 그 길도 혼잡해진다.
- 고정된 이동시간으로 찾은 경로는 그 값에 대한 최단 경로다.
 - 실제 도착 시간을 예측하려면 앞으로의 교통 상황도 고려해야 한다.

- 접근성은 필요한 일자리와 서비스를 이용하기 쉬운 정도다.
 - 도달할 수 있는 목적지의 수와 이동 부담을 함께 본다.
- 도로 연결성은 접근성을 판단하는 한 조건이다.
 - 경로가 있어도 너무 오래 걸리면 이용하기 어려울 수 있다.
- 출발 위치 / 이동 수단 / 목적지의 종류를 정한다.

- 일정 시간 안에 도달할 수 있는 일자리 수를 더한다.
- $A_i(\tau) = \sum_j O_j \times I(T_{ij} \leq \tau)$
- τ 는 허용 시간, i 는 출발지, j 는 목적지다.
- O_j 는 목적지 j 의 일자리 수, T_{ij} 는 최소 이동시간이다.
- $I(\text{조건})$ 는 조건이 참이면 1, 거짓이면 0이다.
- 경로가 없으면 $T_{ij} = \infty$ 로 두어 합계에서 제외한다.

연결 도로와 일자리 접근성

- 도로 X를 추가하기 전과 후의 일자리 접근성을 비교한다.
- B / E / G / H 주변의 일자리 수를 새로 가정한다.

목적지	일자리 수	기본 도시	도로 X 추가 후
B	30	1분	1분
E	20	2분	2분
G	50	4분	4분
H	80	경로 없음	7분
5분 이내 합계	—	100	100
8분 이내 합계	—	100	180

혼잡과 일자리 접근성

- X 를 추가한 도시에서 $F - G$ 의 이동시간이 1분에서 4분으로 늘어난다.
- 도로 연결은 유지되지만 제한 시간 안의 일자리 수는 줄어든다.

목적지	일자리 수	혼잡 전	혼잡 후
B	30	1분	1분
E	20	2분	2분
G	50	4분	7분
H	80	7분	10분
5분 이내 합계	—	100	50
8분 이내 합계	—	180	100

- 도로 셀에서 그래프와 연결 정보를 만든다.
 - 건물의 진입 노드 / 도로의 방향 / 거리와 시간을 저장한다.
- 이동 수요와 처리 용량으로 혼잡을 계산한다.
 - 달라진 이동시간으로 경로와 접근성을 다시 평가한다.
- 접근성이 성장에 미치는 영향은 별도 규칙으로 정한다.

- B와 D를 발견한 직후 *visited*와 *queue*에 무엇이 들어 있는가?
- 간선 수가 가장 적은 경로와 가장 빠른 경로는 언제 일치하는가?
- 혼잡으로 $F - G$ 가 4분이 되면 5분 이내 일자리는 몇 개인가?

- 그래프는 노드와 간선으로 대상 사이의 관계를 표현한다.
- `visited`는 발견 기록 / `queue`는 앞으로 확인할 노드의 순서를 관리한다.
- BFS 알고리즘은 비용이 모두 같을 때 사용할 수 있다.
- 다익스트라 알고리즘은 비용이 음수가 아닐 때 사용할 수 있다.
- A* 알고리즘은 예상 비용을 활용한다.
- 도로 연결이 같아도 혼잡으로 접근성은 낮아질 수 있다.

감사합니다.

utilForever@gmail.com

<https://github.com/utilForever>

Facebook, Twitter, Instagram, LinkedIn: utilForever